Calculus Foundations for Machine Learning and Deep Learning

Akshay Redekar

January 25, 2026

Contents

1 Introduction: Why Calculus for Machine Learning? 2
- 1.1 The Machine Learning Problem . . . 2
- 1.2 The "Best Line" Problem . . . 2

2 Mathematical Formulation 3
- 2.1 Linear Model for Binary Classification . . . 3
- 2.2 Distance from Point to Line . . . 3
- 2.3 Optimization Objective . . . 3

3 Limits: The Foundation of Calculus 4
- 3.1 Concept of Limits . . . 4
- 3.2 One-Sided Limits . . . 4
- 3.3 Limits at Infinity . . . 5

4 Continuity 7
- 4.1 Definition and Properties . . . 7
- 4.2 Important ML Functions . . . 7
- 4.3 Graphical Representation . . . 7

5 Differentiation: Rate of Change 9
- 5.1 Intuitive Understanding . . . 9
- 5.2 Formal Definition . . . 9
- 5.3 Differentiation Rules . . . 9
- 5.4 Common Derivatives (ML Functions) . . . 10

6 Worked Examples: Computing Derivatives 11

7 Finding Maxima and Minima 12
- 7.1 Critical Points . . . 12
- 7.2 First Derivative Test . . . 12
- 7.3 Second Derivative Test . . . 13

8 Application to Machine Learning: Linear Regression **14**
8.1 Problem Setup . . . 14
8.2 Loss Function: Mean Squared Error . . . 14
8.3 Finding the Minimum using Calculus . . . 14
8.4 Numerical Example . . . 15

9 Gradient Descent: The Core ML Algorithm **17**
9.1 Motivation . . . 17
9.2 Algorithm . . . 17
9.3 Visualization: 1D Case . . . 18
9.4 Effect of Learning Rate . . . 18

10 Gradient Descent for Linear Regression **19**
10.1 Setup . . . 19
10.2 Update Rules . . . 19
10.3 Worked Example . . . 19
10.4 Convergence Visualization . . . 20

11 Partial Derivatives and Multivariable Calculus **21**
11.1 Partial Derivatives . . . 21
11.2 The Gradient Vector . . . 21
11.3 Gradient Descent in 2D . . . 22
11.4 Contour Plot Representation . . . 22

12 Advanced Example: Logistic Regression **23**
12.1 Model and Loss Function . . . 23
12.2 Deriving the Gradient . . . 23
12.3 Gradient Descent Updates . . . 24
12.4 Decision Boundary Visualization . . . 24

13 Chain Rule in Neural Networks **25**
13.1 Simple Neural Network . . . 25
13.2 Backpropagation: Computing Gradients . . . 25
13.3 Computational Graph . . . 26

14 Practice Problems **27**

15 Summary and Key Takeaways **28**

16 References and Further Reading **29**

1 Introduction: Why Calculus for Machine Learning?

1.1 The Machine Learning Problem

In machine learning, we aim to find patterns in data and make predictions. Consider a simple binary classification problem:

Binary Classification Setup

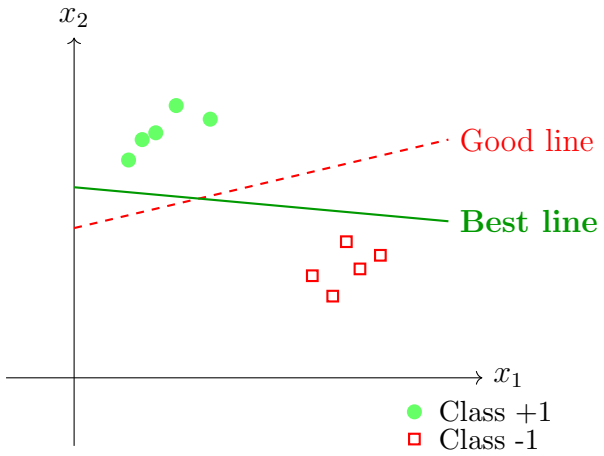
Given: Dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$

- $\mathbf{x}_i \in \mathbb{R}^d$: Feature vector (input)
- $y_i \in \{-1, +1\}$: Class label (output)
- n: Number of training examples
- d: Number of features

Goal: Find a function $f : \mathbb{R}^d \to \{-1, +1\}$ such that:

$$f(\mathbf{x}_i) \approx y_i \quad \text{for all } i \tag{1}$$

1.2 The "Best Line" Problem



Question: How do we mathematically define the "best" line?
Answer: We need to:

1. Define what "best" means (distance from points)
2. Convert this into a mathematical formula (loss function)
3. Find the line that optimizes this formula (using calculus!)

2 Mathematical Formulation

2.1 Linear Model for Binary Classification

Definition 2.1 (Linear Classifier)**.** A linear classifier is a function of the form:

$$f(\mathbf{x}) = \text{sign}(w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b) \tag{2}$$

Or in vector notation:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b) \tag{3}$$

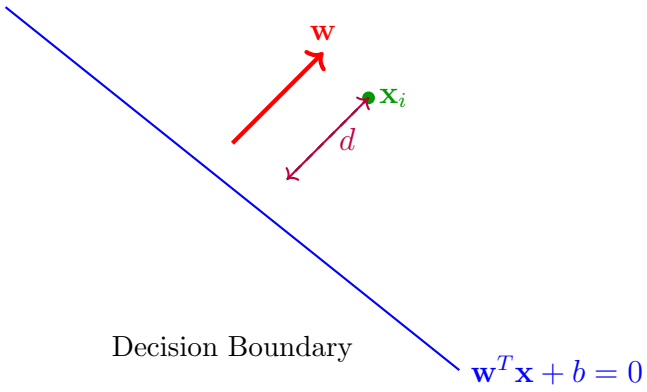
where:

- $\mathbf{w} = [w_1, w_2, \ldots, w_d]^T$: Weight vector
- b: Bias term
- $\text{sign}(z) = \begin{cases} +1 & \text{if } z > 0 \\ -1 & \text{if } z < 0 \end{cases}$

2.2 Distance from Point to Line

For a point $\mathbf{x}_i$ and linear decision boundary $\mathbf{w}^T \mathbf{x} + b = 0$:

$$\text{Distance} = \frac{|\mathbf{w}^T \mathbf{x}_i + b|}{\|\mathbf{w}\|} \tag{4}$$



2.3 Optimization Objective

Goal: Maximize the margin (minimum distance) between classes:

$$\max_{\mathbf{w},b} \left\{ \min_{i=1,\ldots,n} \frac{y_i(\mathbf{w}^T \mathbf{x}_i + b)}{\|\mathbf{w}\|} \right\} \tag{5}$$

Note: We multiply by y_i so that:

- Correctly classified points: $y_i(\mathbf{w}^T \mathbf{x}_i + b) > 0$ (positive distance)
- Misclassified points: $y_i(\mathbf{w}^T \mathbf{x}_i + b) < 0$ (negative distance)

Equivalent formulation: Minimize a loss function

$$\mathcal{L}(\mathbf{w}, b) = \sum_{i=1}^{n} \max(0, -y_i(\mathbf{w}^T \mathbf{x}_i + b)) \tag{6}$$

This is where calculus comes in! We need derivatives to find the optimal $\mathbf{w}$ and b.

3 Limits: The Foundation of Calculus

3.1 Concept of Limits

Definition 3.1 (Limit)**.** The limit of a function $f(x)$ as x approaches a is written as:

$$\lim_{x \to a} f(x) = L \tag{7}$$

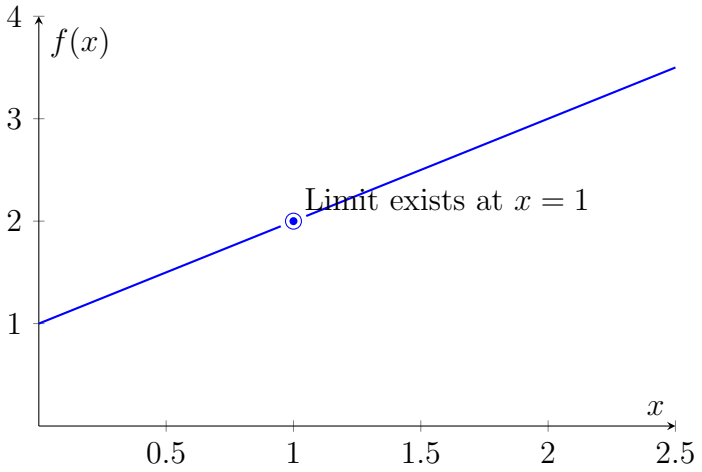
This means: As x gets arbitrarily close to a, $f(x)$ gets arbitrarily close to L.

Example 3.1 (Computing Limits)**.** Consider $f(x) = \frac{x^2-1}{x-1}$ at $x = 1$:

x	0.9	0.99	0.999	1.001	1.01
$f(x)$	1.9	1.99	1.999	2.001	2.01

$$\lim_{x \to 1} \frac{x^2 - 1}{x - 1} = \lim_{x \to 1} \frac{(x-1)(x+1)}{x-1} = \lim_{x \to 1} (x + 1) = 2 \tag{8}$$

Note: $f(1)$ is undefined, but $\lim_{x \to 1} f(x) = 2$



3.2 One-Sided Limits

Definition 3.2 (Right-Hand and Left-Hand Limits)**. Right-hand limit (RHL):**

$$\lim_{x \to a^+} f(x) = \lim_{h \to 0^+} f(a + h) \tag{9}$$

Left-hand limit (LHL):

$$\lim_{x \to a^-} f(x) = \lim_{h \to 0^+} f(a - h) \tag{10}$$

Limit exists if and only if:

$$\lim_{x \to a^-} f(x) = \lim_{x \to a^+} f(x) \tag{11}$$

Example 3.2 (Sign Function - Discontinuous)**.**

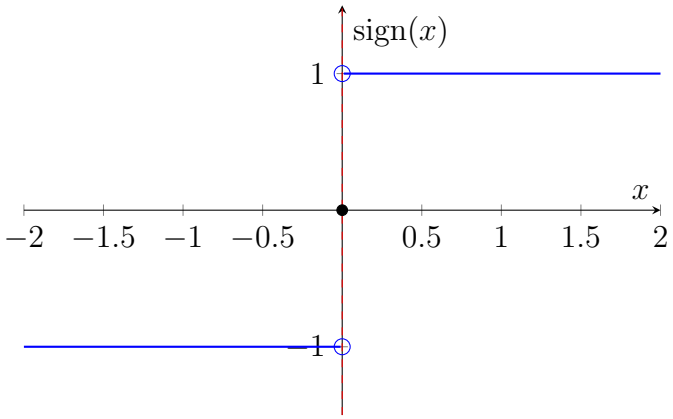
$$f(x) = \text{sign}(x) = \begin{cases} +1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases} \tag{12}$$

At $x = 0$:

$$\lim_{x \to 0^-} \text{sign}(x) = -1 \tag{13}$$

$$\lim_{x \to 0^+} \text{sign}(x) = +1 \tag{14}$$

Since LHL $\neq$ RHL, $\lim_{x \to 0} \text{sign}(x)$ does not exist!



3.3 Limits at Infinity

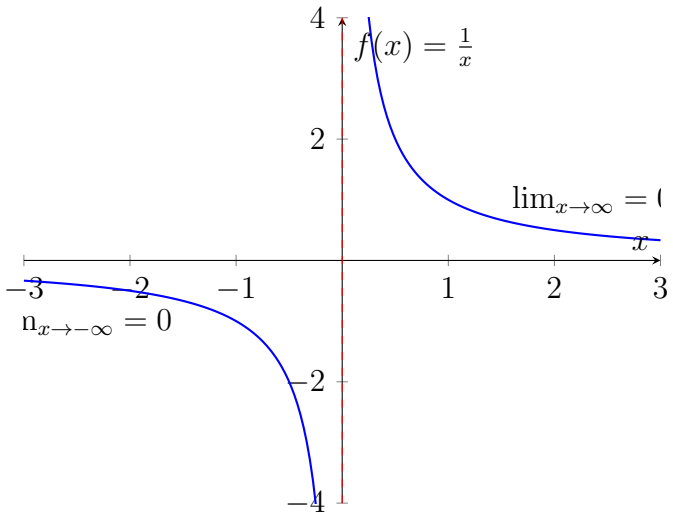
Example 3.3 (Limits Involving Infinity)**.** For $f(x) = \frac{1}{x}$:

$$\lim_{x \to \infty} \frac{1}{x} = 0 \tag{15}$$

$$\lim_{x \to -\infty} \frac{1}{x} = 0 \tag{16}$$

$$\lim_{x \to 0^+} \frac{1}{x} = +\infty \tag{17}$$

$$\lim_{x \to 0^-} \frac{1}{x} = -\infty \tag{18}$$


4
2
−2
−4
−3
−2
−1
1
2
3
$f(x) = \frac{1}{x}$
$\lim_{x\to\infty} =$
x
$\lim_{x\to-\infty} = 0$

4 Continuity

4.1 Definition and Properties

Definition 4.1 (Continuous Function)**.** A function $f(x)$ is **continuous** at $x = a$ if:

$$\lim_{x \to a} f(x) = f(a) \tag{19}$$

Equivalently, all three conditions must hold:

1. $f(a)$ is defined
2. $\lim_{x \to a} f(x)$ exists
3. $\lim_{x \to a} f(x) = f(a)$

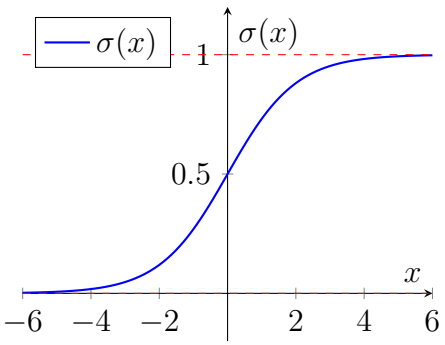
A function is continuous on an interval if it is continuous at every point in that interval.

4.2 Important ML Functions

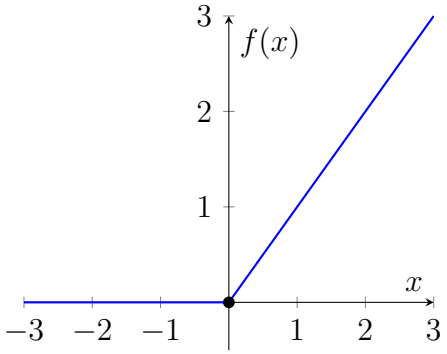
Function	Domain	Range	Formula	Continuous?
Polynomial	$\mathbb{R}$	$\mathbb{R}$	$a_n x^n + \cdots + a_1 x + a_0$	Yes
Exponential	$\mathbb{R}$	$(0, \infty)$	e^x	Yes
Logarithm	$(0, \infty)$	$\mathbb{R}$	$\ln(x)$	Yes
Sigmoid	$\mathbb{R}$	$(0, 1)$	$\sigma(x) = \frac{1}{1+e^{-x}}$	Yes
ReLU	$\mathbb{R}$	$[0, \infty)$	$\max(0, x)$	Yes
Modulus	$\mathbb{R}$	$[0, \infty)$	$\lvert x \rvert$	Yes (not diff. at 0)
Sign	$\mathbb{R}$	$\{-1, 0, 1\}$	$\text{sign}(x)$	No (at 0)

4.3 Graphical Representation

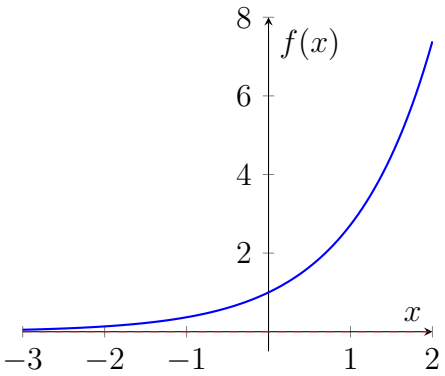
Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$



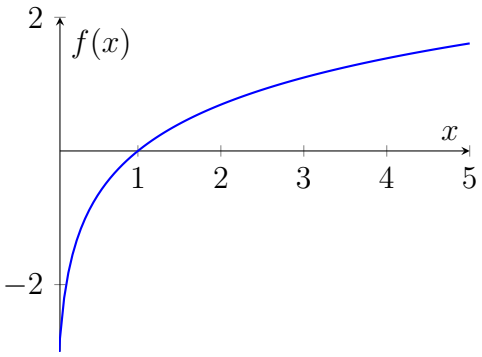
ReLU: $f(x) = \max(0, x)$



Exponential: $f(x) = e^x$



Logarithm: $f(x) = \ln(x)$

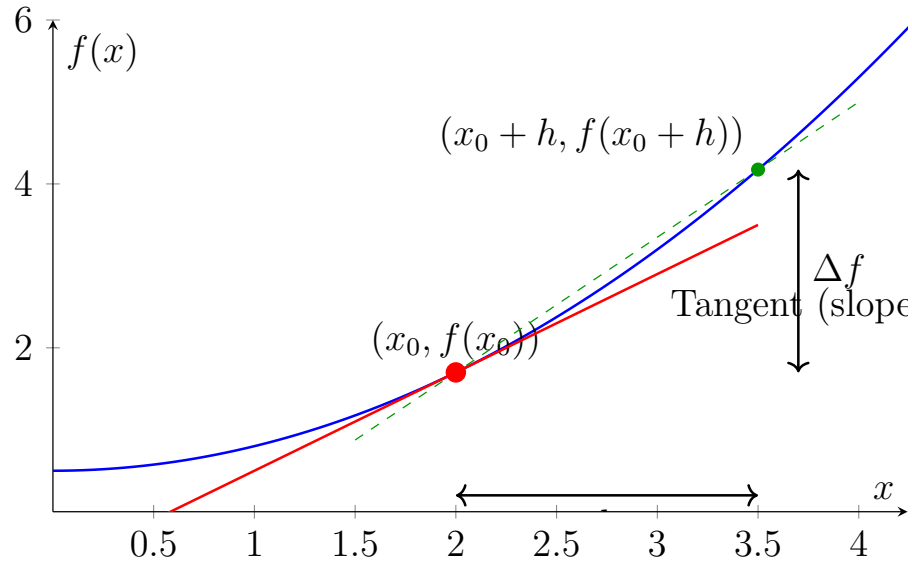


5 Differentiation: Rate of Change

5.1 Intuitive Understanding

What is a Derivative?

Core Question: If I change x by a small amount, how much does $f(x)$ change?
Geometric interpretation: Slope of the tangent line to $f(x)$ at point x
Physical interpretation: Instantaneous rate of change
ML interpretation: How sensitive is the loss function to parameter changes?



5.2 Formal Definition

Definition 5.1 (Derivative)**.** The derivative of $f(x)$ at $x = a$ is:

$$f'(a) = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h} \tag{20}$$

Alternative notation:

$$\left.\frac{df}{dx}\right|_{x=a} = \lim_{\Delta x \to 0} \frac{\Delta f}{\Delta x} = \lim_{\Delta x \to 0} \frac{f(a+\Delta x) - f(a)}{\Delta x} \tag{21}$$

5.3 Differentiation Rules

Essential Differentiation Rules

1. **Power Rule** (Most Important!)

$$\frac{d}{dx}[x^n] = nx^{n-1} \tag{22}$$

2. **Constant Rule**

$$\frac{d}{dx}[c] = 0 \quad \text{(where } c \text{ is constant)} \tag{23}$$

3. Constant Multiple Rule

$$\frac{d}{dx}[cf(x)] = c\frac{d}{dx}[f(x)] \tag{24}$$

4. Sum Rule

$$\frac{d}{dx}[f(x) + g(x)] = \frac{d}{dx}[f(x)] + \frac{d}{dx}[g(x)] \tag{25}$$

5. Product Rule

$$\frac{d}{dx}[f(x)g(x)] = f'(x)g(x) + f(x)g'(x) \tag{26}$$

6. Quotient Rule

$$\frac{d}{dx}\left[\frac{f(x)}{g(x)}\right] = \frac{f'(x)g(x) - f(x)g'(x)}{[g(x)]^2} \tag{27}$$

7. Chain Rule

$$\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x) \tag{28}$$

5.4 Common Derivatives (ML Functions)

Function	$f(x)$	$f'(x)$
Polynomial	x^n	nx^{n-1}
Exponential	e^x	e^x
Natural log	$\ln(x)$	$\frac{1}{x}$
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma(x)(1-\sigma(x))$
Tanh	$\tanh(x)$	$1 - \tanh^2(x)$
ReLU	$\max(0, x)$	$\begin{cases} 1 & x > 0 \\ 0 & x < 0 \\ \text{undef} & x = 0 \end{cases}$
Softplus	$\ln(1 + e^x)$	$\frac{e^x}{1+e^x} = \sigma(x)$

6 Worked Examples: Computing Derivatives

Example 6.1 (Power Rule)**. Problem:** Find $\frac{d}{dx}[x^3]$
Solution: Using power rule with $n = 3$:

$$\frac{d}{dx}[x^3] = 3x^{3-1} = 3x^2 \tag{29}$$

Example 6.2 (Polynomial Function)**. Problem:** Find derivative of $f(x) = 3x^4 - 2x^2 + 5x - 7$
Solution:

$$f'(x) = \frac{d}{dx}[3x^4] - \frac{d}{dx}[2x^2] + \frac{d}{dx}[5x] - \frac{d}{dx}[7] \tag{30}$$
$$= 3(4x^3) - 2(2x) + 5(1) - 0 \tag{31}$$
$$= 12x^3 - 4x + 5 \tag{32}$$

Example 6.3 (Chain Rule with Sigmoid)**. Problem:** Find derivative of $f(x) = \sigma(2x+3)$ where $\sigma(z) = \frac{1}{1+e^{-z}}$
Solution: Let $z = 2x + 3$, so $f(x) = \sigma(z)$
Using chain rule:

$$f'(x) = \frac{d\sigma}{dz} \cdot \frac{dz}{dx} \tag{33}$$
$$= \sigma(z)(1 - \sigma(z)) \cdot 2 \tag{34}$$
$$= 2\sigma(2x + 3)(1 - \sigma(2x + 3)) \tag{35}$$

Example 6.4 (Product Rule)**. Problem:** Find derivative of $f(x) = x^2 e^x$
Solution:

$$f'(x) = \frac{d}{dx}[x^2] \cdot e^x + x^2 \cdot \frac{d}{dx}[e^x] \tag{36}$$
$$= 2x \cdot e^x + x^2 \cdot e^x \tag{37}$$
$$= e^x(2x + x^2) \tag{38}$$
$$= xe^x(2 + x) \tag{39}$$

Example 6.5 (Quotient Rule)**. Problem:** Find derivative of $f(x) = \frac{x^2+1}{x-1}$
Solution:

$$f'(x) = \frac{(2x)(x-1) - (x^2+1)(1)}{(x-1)^2} \tag{40}$$
$$= \frac{2x^2 - 2x - x^2 - 1}{(x-1)^2} \tag{41}$$
$$= \frac{x^2 - 2x - 1}{(x-1)^2} \tag{42}$$

7 Finding Maxima and Minima

7.1 Critical Points

Definition 7.1 (Critical Point)**.** A point $x = c$ is a **critical point** of $f(x)$ if:

$$f'(c) = 0 \quad \text{or} \quad f'(c) \text{ does not exist} \tag{43}$$

Theorem 7.1 (Fermat's Theorem)**.** If f has a local maximum or minimum at $x = c$, and $f'(c)$ exists, then:

$$f'(c) = 0 \tag{44}$$

7.2 First Derivative Test

First Derivative Test

Let c be a critical point of f (i.e., $f'(c) = 0$).

- If $f'(x)$ changes from **positive to negative** at c:

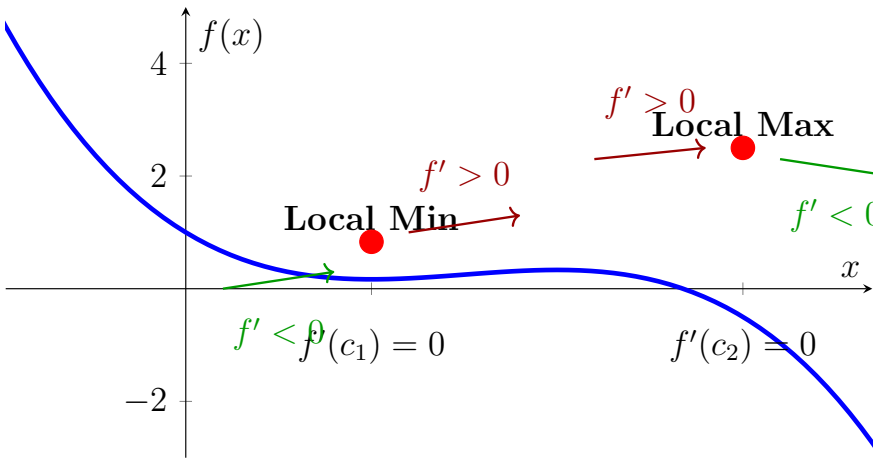
$$c \text{ is a } \textbf{local maximum} \tag{45}$$

- If $f'(x)$ changes from **negative to positive** at c:

$$c \text{ is a } \textbf{local minimum} \tag{46}$$

- If $f'(x)$ does not change sign at c:

$$c \text{ is } \textbf{neither} \text{ (inflection point)} \tag{47}$$



7.3 Second Derivative Test

Second Derivative Test

Let c be a critical point where $f'(c) = 0$.

- If $f''(c) > 0$ (concave up):

$$c \text{ is a } \textbf{local minimum} \tag{48}$$

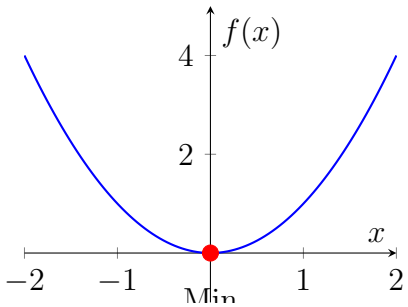
- If $f''(c) < 0$ (concave down):

$$c \text{ is a } \textbf{local maximum} \tag{49}$$

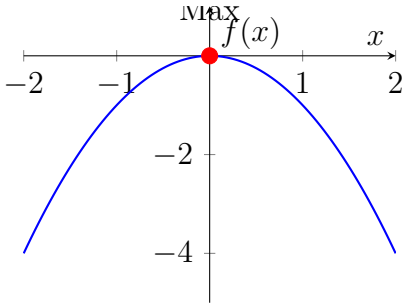
- If $f''(c) = 0$:

$$\text{Test is } \textbf{inconclusive} \tag{50}$$

Concave Up ($f'' > 0$): Minimum



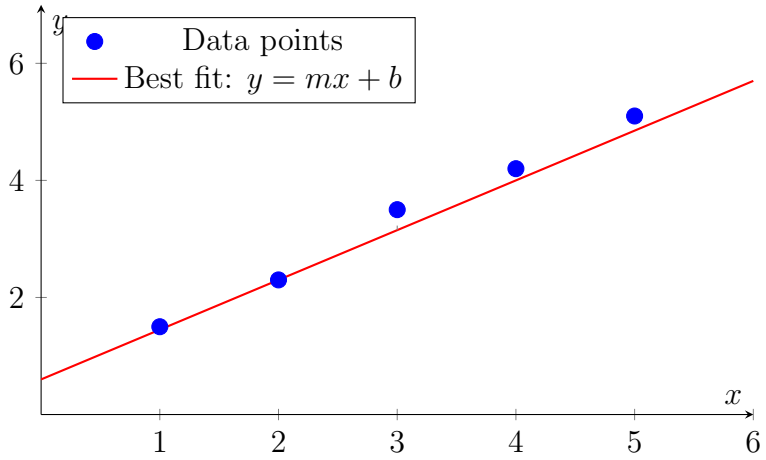
Concave Down ($f'' < 0$): Maximum



8 Application to Machine Learning: Linear Regression

8.1 Problem Setup

Problem 8.1 (Simple Linear Regression)**.** Given dataset: $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$
Model: $\hat{y} = mx + b$
Goal: Find m (slope) and b (intercept) that best fit the data



8.2 Loss Function: Mean Squared Error

Definition 8.1 (MSE Loss)**.** The Mean Squared Error measures average squared difference between predictions and actual values:

$$\mathcal{L}(m, b) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - (mx_i + b))^2 \tag{51}$$

Goal: Minimize $\mathcal{L}(m, b)$ by finding optimal m^* and b^*

8.3 Finding the Minimum using Calculus

To find minimum, we set partial derivatives to zero:

$$\frac{\partial \mathcal{L}}{\partial m} = 0 \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial b} = 0 \tag{52}$$

Step 1: Compute $\frac{\partial \mathcal{L}}{\partial m}$

$$\frac{\partial \mathcal{L}}{\partial m} = \frac{\partial}{\partial m}\left[\frac{1}{n}\sum_{i=1}^{n}(y_i - mx_i - b)^2\right] \tag{53}$$

$$= \frac{1}{n}\sum_{i=1}^{n} 2(y_i - mx_i - b)(-x_i) \tag{54}$$

$$= -\frac{2}{n}\sum_{i=1}^{n} x_i(y_i - mx_i - b) \tag{55}$$

Step 2: Compute $\frac{\partial \mathcal{L}}{\partial b}$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial}{\partial b}\left[\frac{1}{n}\sum_{i=1}^{n}(y_i - mx_i - b)^2\right] \tag{56}$$

$$= \frac{1}{n}\sum_{i=1}^{n} 2(y_i - mx_i - b)(-1) \tag{57}$$

$$= -\frac{2}{n}\sum_{i=1}^{n}(y_i - mx_i - b) \tag{58}$$

Step 3: Set derivatives to zero and solve
From $\frac{\partial \mathcal{L}}{\partial b} = 0$:

$$\sum_{i=1}^{n}(y_i - mx_i - b) = 0 \implies b = \bar{y} - m\bar{x} \tag{59}$$

where $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ and $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$
From $\frac{\partial \mathcal{L}}{\partial m} = 0$ (substituting b):

$$m = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2} \tag{60}$$

8.4 Numerical Example

Example 8.1 (Computing Best Fit Line)**. Data:** $(1, 2), (2, 3), (3, 5), (4, 6)$
Step 1: Compute means

$$\bar{x} = \frac{1 + 2 + 3 + 4}{4} = 2.5 \tag{61}$$

$$\bar{y} = \frac{2 + 3 + 5 + 6}{4} = 4 \tag{62}$$

Step 2: Compute slope m

i	x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$
1	1	2	-1.5	-2	3.0
2	2	3	-0.5	-1	0.5
3	3	5	0.5	1	0.5
4	4	6	1.5	2	3.0
				Sum:	**7.0**

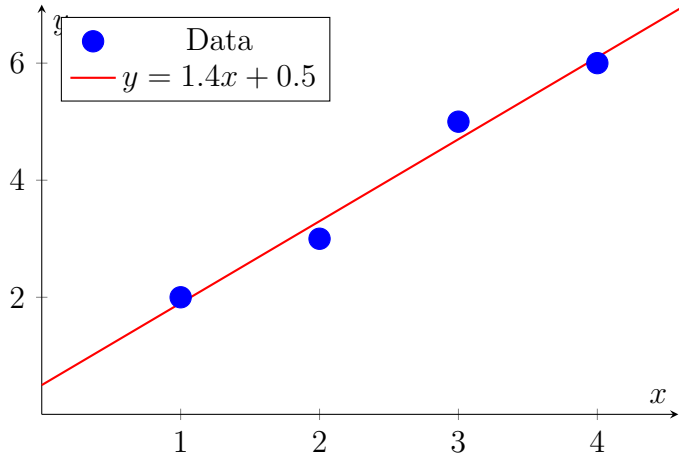
$$\sum_{i=1}^{4}(x_i - \bar{x})^2 = (-1.5)^2 + (-0.5)^2 + (0.5)^2 + (1.5)^2 = 5.0 \tag{63}$$

$$m = \frac{7.0}{5.0} = 1.4 \tag{64}$$

Step 3: Compute intercept b

$$b = \bar{y} - m\bar{x} = 4 - 1.4(2.5) = 4 - 3.5 = 0.5 \tag{65}$$

Result: Best fit line is $y = 1.4x + 0.5$


Data
$y = 1.4x + 0.5$
y
x
2
4
6
1
2
3
4

9 Gradient Descent: The Core ML Algorithm

9.1 Motivation

For complex models (like neural networks), we cannot solve $\nabla \mathcal{L} = 0$ analytically.

Solution: Use iterative optimization!

Gradient Descent Intuition

Analogy: Imagine standing on a mountain in fog. You want to reach the valley (minimum).
Strategy:

1. Look around and find the direction of steepest descent
2. Take a small step in that direction
3. Repeat until you reach the bottom

The **gradient** points in the direction of steepest *ascent*, so we go in the **opposite direction**!

9.2 Algorithm

Gradient Descent Algorithm

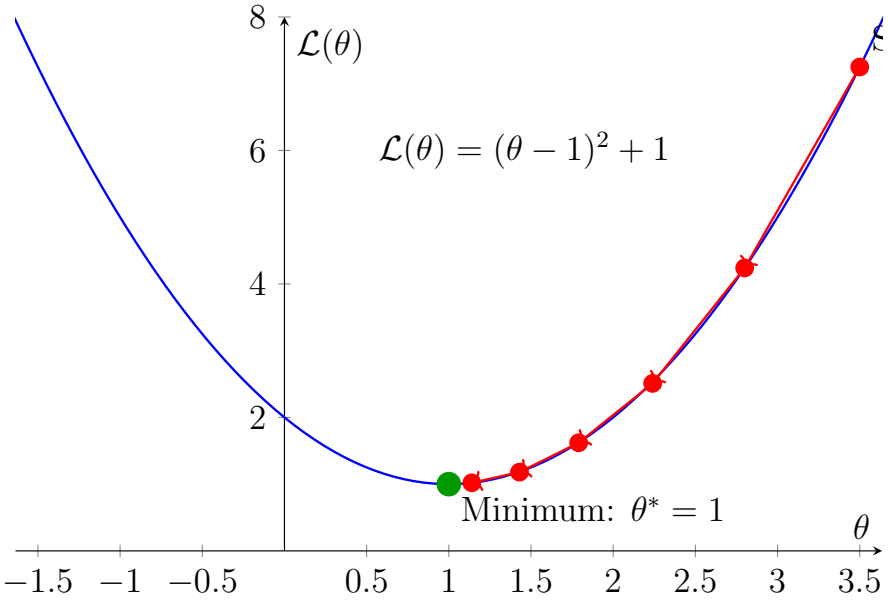
Goal: Minimize $\mathcal{L}(\theta)$ where θ represents parameters
Initialize: Choose random θ_0
Iterate: For $t = 0, 1, 2, \ldots$ until convergence:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t) \tag{66}$$

where:

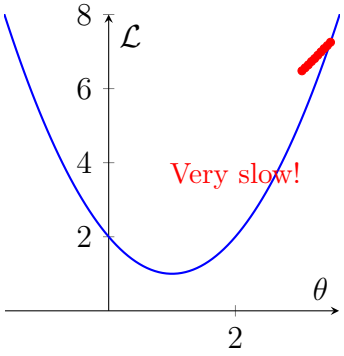
- $\eta > 0$: Learning rate (step size)
- $\nabla_\theta \mathcal{L}$: Gradient of loss w.r.t. parameters
- Negative sign: move in direction opposite to gradient

9.3 Visualization: 1D Case

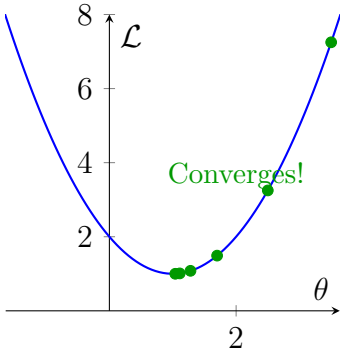


9.4 Effect of Learning Rate

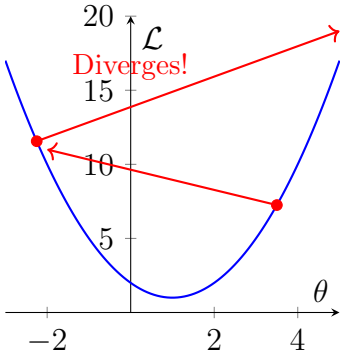
Too Small Learning Rate ($\eta = 0.01$)



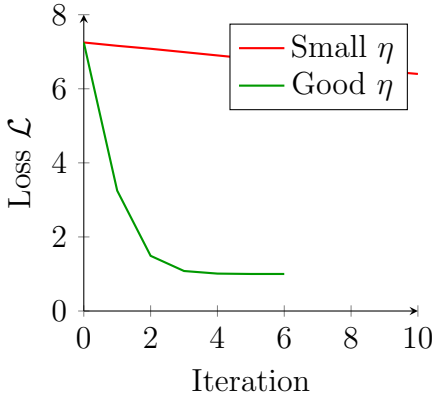
Good Learning Rate ($\eta = 0.2$)



Too Large Learning Rate ($\eta = 1.5$)



Loss vs Iterations



10 Gradient Descent for Linear Regression

10.1 Setup

Loss function:

$$\mathcal{L}(m,b) = \frac{1}{n}\sum_{i=1}^{n}(y_i - (mx_i + b))^2 \tag{67}$$

Parameters: $\theta = [m, b]^T$
Gradients:

$$\frac{\partial \mathcal{L}}{\partial m} = -\frac{2}{n}\sum_{i=1}^{n} x_i(y_i - mx_i - b) \tag{68}$$

$$\frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - mx_i - b) \tag{69}$$

10.2 Update Rules

$$m_{t+1} = m_t - \eta \left.\frac{\partial \mathcal{L}}{\partial m}\right|_{m=m_t, b=b_t} \tag{70}$$

$$b_{t+1} = b_t - \eta \left.\frac{\partial \mathcal{L}}{\partial b}\right|_{m=m_t, b=b_t} \tag{71}$$

10.3 Worked Example

Example 10.1 (Gradient Descent on Simple Data)**. Data:** $(1,2), (2,4), (3,6)$
True line: $y = 2x$ (we want to recover this!)
Initialize: $m_0 = 0, b_0 = 0, \eta = 0.1$
Iteration 1:
Predictions: $\hat{y}_i = m_0 x_i + b_0 = 0$
Errors: $y_1 - \hat{y}_1 = 2, y_2 - \hat{y}_2 = 4, y_3 - \hat{y}_3 = 6$

$$\left.\frac{\partial \mathcal{L}}{\partial m}\right|_{m_0,b_0} = -\frac{2}{3}(1\cdot 2 + 2\cdot 4 + 3\cdot 6) = -\frac{2}{3}(28) = -18.67 \tag{72}$$

$$\left.\frac{\partial \mathcal{L}}{\partial b}\right|_{m_0,b_0} = -\frac{2}{3}(2 + 4 + 6) = -\frac{2}{3}(12) = -8 \tag{73}$$

Updates:

$$m_1 = 0 - 0.1(-18.67) = 1.867 \tag{74}$$

$$b_1 = 0 - 0.1(-8) = 0.8 \tag{75}$$

Iteration 2:
Predictions: $\hat{y}_i = 1.867 x_i + 0.8$
$\hat{y}_1 = 2.667, \hat{y}_2 = 4.534, \hat{y}_3 = 6.401$
Errors: $y_1 - \hat{y}_1 = -0.667, y_2 - \hat{y}_2 = -0.534, y_3 - \hat{y}_3 = -0.401$

$$\left.\frac{\partial \mathcal{L}}{\partial m}\right|_{m_1,b_1} = -\frac{2}{3}(1(-0.667) + 2(-0.534) + 3(-0.401)) = 1.738 \tag{76}$$

$$\left.\frac{\partial \mathcal{L}}{\partial b}\right|_{m_1,b_1} = -\frac{2}{3}(-0.667 - 0.534 - 0.401) = 1.069 \tag{77}$$

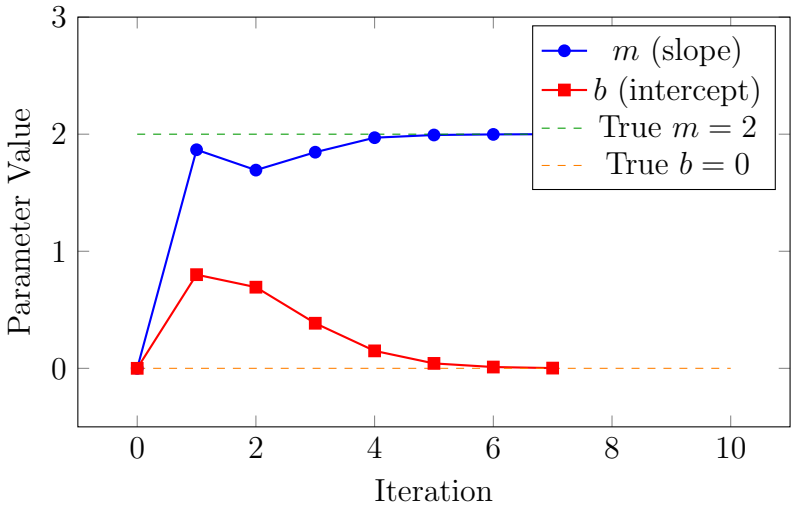
Updates:

$$m_2 = 1.867 - 0.1(1.738) = 1.693 \tag{78}$$

$$b_2 = 0.8 - 0.1(1.069) = 0.693 \tag{79}$$

Continuing this process, we converge to $m \approx 2, b \approx 0$!

10.4 Convergence Visualization



11 Partial Derivatives and Multivariable Calculus

11.1 Partial Derivatives

Definition 11.1 (Partial Derivative)**.** For a function $f(x, y)$ of two variables, the partial derivative with respect to x is:

$$\frac{\partial f}{\partial x} = \lim_{h \to 0} \frac{f(x+h, y) - f(x, y)}{h} \tag{80}$$

Interpretation: Rate of change of f as x varies, holding y constant.

Example 11.1 (Computing Partial Derivatives)**. Function:** $f(x, y) = x^2y + 3xy^2 - 5$
Partial derivative w.r.t. x (treat y as constant):

$$\frac{\partial f}{\partial x} = 2xy + 3y^2 \tag{81}$$

Partial derivative w.r.t. y (treat x as constant):

$$\frac{\partial f}{\partial y} = x^2 + 6xy \tag{82}$$

11.2 The Gradient Vector

Definition 11.2 (Gradient)**.** For a scalar function $f : \mathbb{R}^n \to \mathbb{R}$, the gradient is:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \tag{83}$$

The gradient points in the direction of steepest ascent.

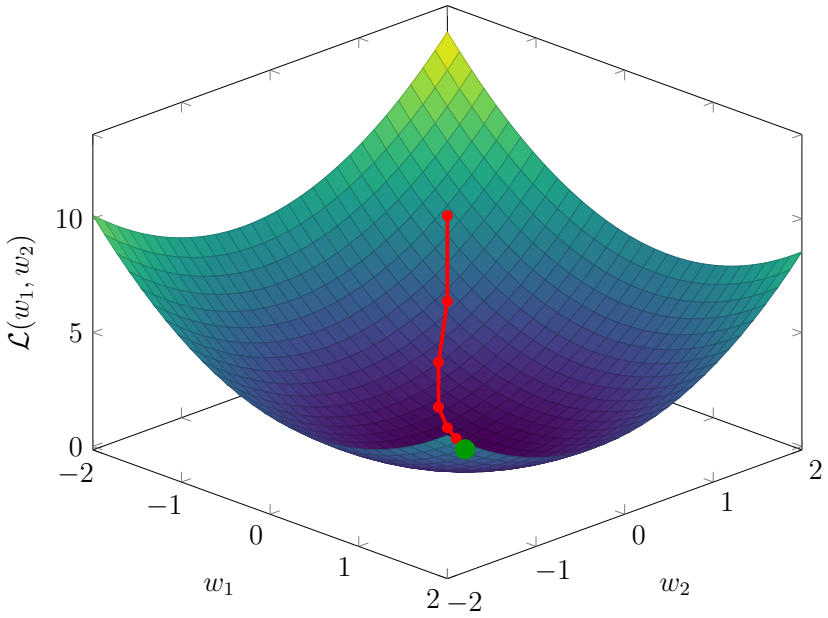
Example 11.2 (Gradient for Loss Function)**.** For MSE loss with parameters $\mathbf{w} = [w_1, w_2]^T$:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - (w_1 x_{i1} + w_2 x_{i2}))^2 \tag{84}$$

Gradient:

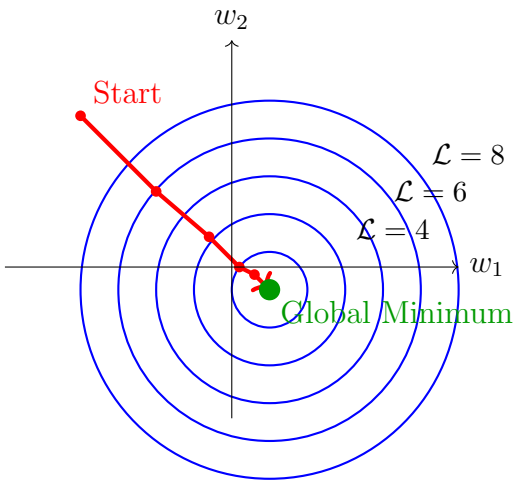
$$\nabla_{\mathbf{w}} \mathcal{L} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial w_1} \\ \frac{\partial \mathcal{L}}{\partial w_2} \end{bmatrix} = -\frac{2}{n} \sum_{i=1}^{n} (y_i - \mathbf{w}^T \mathbf{x}_i) \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix} \tag{85}$$

11.3 Gradient Descent in 2D



11.4 Contour Plot Representation

Contour plot showing gradient descent path from starting point to global minimum. Each contour line represents points with the same loss value.



12 Advanced Example: Logistic Regression

12.1 Model and Loss Function

Logistic Regression for Binary Classification

Model:

$$P(y=1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}+b) = \frac{1}{1+e^{-(\mathbf{w}^T\mathbf{x}+b)}} \tag{86}$$

Binary Cross-Entropy Loss:

$$\mathcal{L}(\mathbf{w},b) = -\frac{1}{n}\sum_{i=1}^{n}[y_i\log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)] \tag{87}$$

where $\hat{y}_i = \sigma(\mathbf{w}^T\mathbf{x}_i+b)$ and $y_i \in \{0,1\}$

12.2 Deriving the Gradient

Key derivative:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1-\sigma(z)) \tag{88}$$

Gradient w.r.t. w:
Let $z_i = \mathbf{w}^T\mathbf{x}_i + b$

$$\frac{\partial\mathcal{L}}{\partial w_j} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i\frac{1}{\hat{y}_i}\frac{\partial\hat{y}_i}{\partial w_j} + (1-y_i)\frac{-1}{1-\hat{y}_i}\frac{\partial\hat{y}_i}{\partial w_j}\right] \tag{89}$$

$$= -\frac{1}{n}\sum_{i=1}^{n}\left[y_i\frac{\sigma'(z_i)}{\sigma(z_i)} - (1-y_i)\frac{\sigma'(z_i)}{1-\sigma(z_i)}\right]x_{ij} \tag{90}$$

$$= -\frac{1}{n}\sum_{i=1}^{n}\left[y_i(1-\sigma(z_i)) - (1-y_i)\sigma(z_i)\right]x_{ij} \tag{91}$$

$$= -\frac{1}{n}\sum_{i=1}^{n}(y_i-\sigma(z_i))x_{ij} \tag{92}$$

$$= -\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)x_{ij} \tag{93}$$

In vector form:

$$\nabla_{\mathbf{w}}\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)\mathbf{x}_i \tag{94}$$

Similarly for bias:

$$\frac{\partial\mathcal{L}}{\partial b} = -\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i) \tag{95}$$

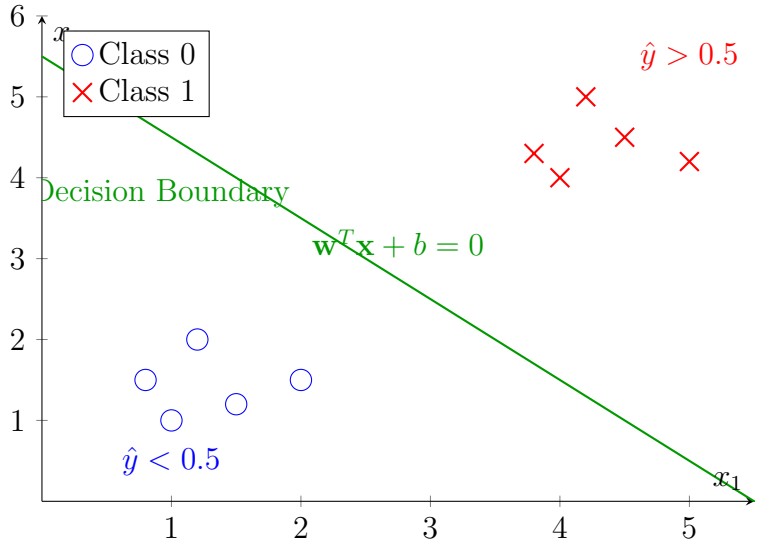
12.3 Gradient Descent Updates

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \eta \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i^{(t)}) \mathbf{x}_i \tag{96}$$

$$b_{t+1} = b_t + \eta \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i^{(t)}) \tag{97}$$

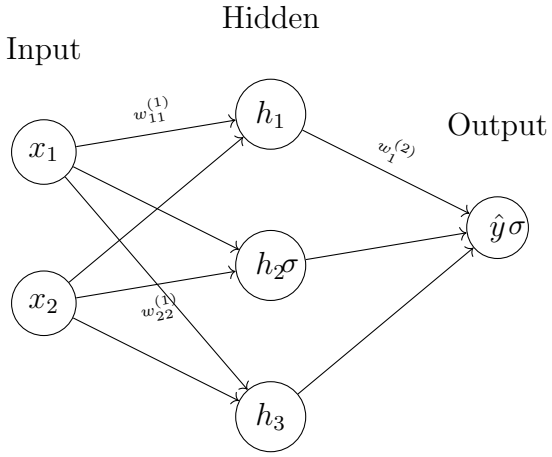
where $\hat{y}_i^{(t)} = \sigma(\mathbf{w}_t^T \mathbf{x}_i + b_t)$

12.4 Decision Boundary Visualization



13 Chain Rule in Neural Networks

13.1 Simple Neural Network



Forward pass:

$$z_j^{(1)} = \sum_i w_{ji}^{(1)} x_i + b_j^{(1)} \tag{98}$$

$$h_j = \sigma(z_j^{(1)}) \tag{99}$$

$$z^{(2)} = \sum_j w_j^{(2)} h_j + b^{(2)} \tag{100}$$

$$\hat{y} = \sigma(z^{(2)}) \tag{101}$$

13.2 Backpropagation: Computing Gradients

Loss: $\mathcal{L} = (\hat{y} - y)^2$

Goal: Compute $\frac{\partial \mathcal{L}}{\partial w_j^{(2)}}$ and $\frac{\partial \mathcal{L}}{\partial w_{ji}^{(1)}}$

For output layer:

$$\frac{\partial \mathcal{L}}{\partial w_j^{(2)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial w_j^{(2)}} \tag{102}$$

$$= 2(\hat{y} - y) \cdot \sigma'(z^{(2)}) \cdot h_j \tag{103}$$

$$= 2(\hat{y} - y) \cdot \hat{y}(1 - \hat{y}) \cdot h_j \tag{104}$$

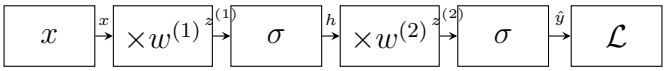
For hidden layer (using chain rule):

$$\frac{\partial \mathcal{L}}{\partial w_{ji}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial h_j} \cdot \frac{\partial h_j}{\partial z_j^{(1)}} \cdot \frac{\partial z_j^{(1)}}{\partial w_{ji}^{(1)}} \tag{105}$$

$$= 2(\hat{y} - y) \cdot \hat{y}(1 - \hat{y}) \cdot w_j^{(2)} \cdot h_j(1 - h_j) \cdot x_i \tag{106}$$

13.3 Computational Graph

Forward Pass



$$\frac{\partial \mathcal{L}}{\partial x} \longrightarrow \frac{\partial \mathcal{L}}{\partial z^{(1)}} \longrightarrow \frac{\partial \mathcal{L}}{\partial h} \longrightarrow \frac{\partial \mathcal{L}}{\partial z^{(2)}} \longrightarrow \frac{\partial \mathcal{L}}{\partial \hat{y}}$$

Backward Pass (Gradients)

14 Practice Problems

Problem 14.1. Compute the following limits:

(a) $\lim_{x\to 2} \frac{x^2-4}{x-2}$

(b) $\lim_{x\to 0} \frac{\sin(x)}{x}$

(c) $\lim_{x\to\infty} \frac{3x^2+2x}{x^2-1}$

Problem 14.2. Find the derivatives of:

(a) $f(x) = 5x^4 - 3x^2 + 7x - 2$

(b) $g(x) = e^{2x}$

(c) $h(x) = \ln(x^2 + 1)$

(d) $k(x) = x^2 \sin(x)$

Problem 14.3. Find the critical points and classify them as maxima, minima, or neither:

(a) $f(x) = x^3 - 3x + 2$

(b) $g(x) = 2x^3 - 9x^2 + 12x$

Problem 14.4. For the function $f(x, y) = x^2 + 2xy + y^2 - 4x - 6y$:

(a) Find $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$

(b) Find the critical point(s)

(c) Determine if it's a maximum, minimum, or saddle point

Problem 14.5. Given data points $(0, 1), (1, 3), (2, 2), (3, 5)$:

(a) Set up the MSE loss function for linear regression

(b) Compute the gradients $\frac{\partial \mathcal{L}}{\partial m}$ and $\frac{\partial \mathcal{L}}{\partial b}$ at $m = 1, b = 1$

(c) Perform one gradient descent update with $\eta = 0.1$

Problem 14.6. For sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$:

(a) Prove that $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

(b) Compute $\sigma(0), \sigma(2), \sigma(-2)$

(c) Find $\lim_{x\to\infty} \sigma(x)$ and $\lim_{x\to-\infty} \sigma(x)$

Problem 14.7. Implement one iteration of gradient descent for logistic regression with:

- Data: $(\mathbf{x}_1, y_1) = ([1, 2]^T, 1)$ and $(\mathbf{x}_2, y_2) = ([3, 1]^T, 0)$
- Initial: $\mathbf{w}_0 = [0, 0]^T, b_0 = 0$
- Learning rate: $\eta = 0.5$

15 Summary and Key Takeaways

Essential Calculus Concepts for ML

1. **Limits:**
   - Foundation of calculus
   - Describes behavior of functions at boundaries
   - Used to define continuity and derivatives
2. **Derivatives:**
   - Measure rate of change
   - Slope of tangent line
   - Key for optimization in ML
3. **Gradient:**
   - Vector of partial derivatives
   - Points in direction of steepest ascent
   - Used in gradient descent: move opposite to gradient
4. **Optimization:**
   - Finding maxima/minima: set derivative to zero
   - Gradient descent: iterative optimization
   - Learning rate: controls step size
5. **Chain Rule:**
   - Composing derivatives
   - Essential for backpropagation in neural networks
   - Allows computing gradients through complex functions

ML Applications

Linear Regression:

$$\min_{m,b} \frac{1}{n} \sum_{i=1}^{n} (y_i - (mx_i + b))^2 \tag{107}$$

Logistic Regression:

$$\min_{\mathbf{w},b} -\frac{1}{n} \sum_{i=1}^{n} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \tag{108}$$

Neural Networks:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t) \tag{109}$$

All solved using calculus and gradient descent!

16 References and Further Reading

1. **Calculus:** James Stewart, "Calculus: Early Transcendentals"
2. **ML Mathematics:** Deisenroth, Faisal, Ong, "Mathematics for Machine Learning"
3. **Deep Learning:** Goodfellow, Bengio, Courville, "Deep Learning"
4. **Khan Academy:** Calculus and Multivariable Calculus courses
5. **3Blue1Brown:** "Essence of Calculus" video series

Vector Spaces in Machine Learning and Deep Learning
Real-World Examples with Detailed Solutions

Akshay Redekar

January 25, 2026

Contents

1 Principal Component Analysis (PCA) for Image Compression 2

2 Word Embeddings and Vector Arithmetic 4

3 Linear Independence in Feature Engineering 6

4 Dimensionality and Curse of Dimensionality 8

5 Neural Network Parameter Space 10

6 Cosine Similarity in Recommender Systems 12

7 Subspaces in Image Processing 14

8 Orthogonality in Signal Processing 16

9 Rank and Column Space in Data Analysis 18

10 Gram-Schmidt Orthogonalization in QR Decomposition 20

11 Kernel Methods and Implicit Feature Spaces 22

12 Batch Normalization and Affine Transformations 24

13 Additional Practice Problems 26

14 Summary of Vector Space Concepts in ML/DL 26

1 Principal Component Analysis (PCA) for Image Compression

Problem 1.1 (MNIST Digit Compression)**.** The MNIST dataset contains 28×28 grayscale images of handwritten digits (0-9). Each image can be represented as a vector in $\mathbb{R}^{784}$ (since $28 \times 28 = 784$ pixels).

Given: A dataset of 60,000 training images where each image $\mathbf{x}_i \in \mathbb{R}^{784}$.

Task: Use PCA to reduce dimensionality from 784 to 50 while preserving maximum variance.

Questions:

1. What percentage of variance is preserved?
2. How do you compute the principal components?
3. How do you reconstruct an approximation of the original image?

Solution 1.1 (Step-by-Step PCA Implementation)**. Step 1: Center the data**

Compute the mean image:

$$\boldsymbol{\mu} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i \tag{1}$$

Center each image:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i - \boldsymbol{\mu} \tag{2}$$

Form the centered data matrix:

$$X = \begin{bmatrix} \tilde{\mathbf{x}}_1^T \\ \tilde{\mathbf{x}}_2^T \\ \vdots \\ \tilde{\mathbf{x}}_n^T \end{bmatrix} \in \mathbb{R}^{60000\times 784} \tag{3}$$

Step 2: Compute covariance matrix

$$\Sigma = \frac{1}{n-1} X^T X \in \mathbb{R}^{784\times 784} \tag{4}$$

Step 3: Eigenvalue decomposition

Solve:

$$\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i \tag{5}$$

Sort eigenvalues: $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_{784}$

Corresponding eigenvectors: $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_{784}$

Step 4: Select top k=50 components

Form projection matrix:

$$W = [\mathbf{v}_1 \mid \mathbf{v}_2 \mid \cdots \mid \mathbf{v}_{50}] \in \mathbb{R}^{784\times 50} \tag{6}$$

Step 5: Transform data to lower dimension

For each image:

$$\mathbf{z}_i = W^T \tilde{\mathbf{x}}_i \in \mathbb{R}^{50} \tag{7}$$

Step 6: Compute variance preserved

$$\text{Variance Preserved} = \frac{\sum_{i=1}^{50} \lambda_i}{\sum_{i=1}^{784} \lambda_i} \times 100\% \tag{8}$$

Typical result: With 50 components, approximately 85-90% variance is preserved.

Step 7: Reconstruction

To reconstruct approximation of original image:

$$\hat{\mathbf{x}}_i = \boldsymbol{\mu} + W\mathbf{z}_i \tag{9}$$

Reconstruction error:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \tag{10}$$

Numerical Example

Sample calculation with toy data (3 images, 4 pixels each):

Original images:

$$X = \begin{bmatrix} 200 & 150 & 100 & 50 \\ 210 & 160 & 110 & 60 \\ 190 & 140 & 90 & 40 \end{bmatrix} \tag{11}$$

Mean: $\boldsymbol{\mu} = [200, 150, 100, 50]^T$

Centered data:

$$\tilde{X} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 10 & 10 & 10 & 10 \\ -10 & -10 & -10 & -10 \end{bmatrix} \tag{12}$$

Covariance matrix:

$$\Sigma = \frac{1}{2}\tilde{X}^T\tilde{X} = \begin{bmatrix} 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 \end{bmatrix} \tag{13}$$

Eigenvalues: $\lambda_1 = 400, \lambda_2 = \lambda_3 = \lambda_4 = 0$

First eigenvector: $\mathbf{v}_1 = \frac{1}{2}[1, 1, 1, 1]^T$

Variance preserved with k=1: $\frac{400}{400} = 100\%$

2 Word Embeddings and Vector Arithmetic

Problem 2.1 (Word2Vec Analogy: King - Man + Woman = Queen)**.** In Word2Vec embeddings trained on Google News corpus (3 billion words), words are represented as 300-dimensional vectors.

Given vectors (simplified to 3D for illustration):

$$\mathbf{v}_{\text{king}} = [9, 0, 9.5] \tag{14}$$
$$\mathbf{v}_{\text{man}} = [7, 0, 5] \tag{15}$$
$$\mathbf{v}_{\text{woman}} = [7, 10, 5] \tag{16}$$
$$\mathbf{v}_{\text{queen}} = [9, 10, 9.5] \tag{17}$$

where dimensions represent: [wealth, femininity, power]

Task: Verify the analogy "King is to Man as Queen is to Woman" using vector arithmetic.

Solution 2.1 (Vector Arithmetic for Analogies)**. Step 1: Compute the relationship vector**

The relationship from "man" to "king" (adding royalty):

$$\mathbf{r} = \mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} \tag{18}$$

$$\mathbf{r} = [9, 0, 9.5] - [7, 0, 5] \tag{19}$$
$$= [2, 0, 4.5] \tag{20}$$

This vector represents "royalty" concept.

Step 2: Apply relationship to "woman"

$$\mathbf{v}_{\text{result}} = \mathbf{v}_{\text{woman}} + \mathbf{r} \tag{21}$$

$$\mathbf{v}_{\text{result}} = [7, 10, 5] + [2, 0, 4.5] \tag{22}$$
$$= [9, 10, 9.5] \tag{23}$$

Step 3: Find nearest neighbor using cosine similarity

For any word vector $\mathbf{w}$ in vocabulary:

$$\text{similarity}(\mathbf{v}_{\text{result}}, \mathbf{w}) = \frac{\mathbf{v}_{\text{result}} \cdot \mathbf{w}}{\|\mathbf{v}_{\text{result}}\|\|\mathbf{w}\|} \tag{24}$$

Compare with "queen":

$$\mathbf{v}_{\text{result}} \cdot \mathbf{v}_{\text{queen}} = 9(9) + 10(10) + 9.5(9.5) \tag{25}$$
$$= 81 + 100 + 90.25 = 271.25 \tag{26}$$

$$\|\mathbf{v}_{\text{result}}\| = \sqrt{9^2 + 10^2 + 9.5^2} = \sqrt{271.25} \approx 16.47 \tag{27}$$
$$\|\mathbf{v}_{\text{queen}}\| = \sqrt{9^2 + 10^2 + 9.5^2} = \sqrt{271.25} \approx 16.47 \tag{28}$$

$$\text{cosine similarity} = \frac{271.25}{16.47 \times 16.47} = \frac{271.25}{271.26} \approx 0.9999 \tag{29}$$

Result: The cosine similarity is nearly 1.0, confirming that:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}} \tag{30}$$

Problem 2.2 (Real-World Word2Vec Calculation)**.** Using actual Word2Vec embeddings (300-dimensional), compute:

$$\text{Paris} - \text{France} + \text{Germany} = ? \tag{31}$$

Expected result: Berlin

Solution 2.2 (Geographic Analogy)**. Given:** Word2Vec vectors from Google News
Process:

1. Load pre-trained vectors
2. Extract vectors:

$$\mathbf{v}_{\text{Paris}} \in \mathbb{R}^{300} \tag{32}$$
$$\mathbf{v}_{\text{France}} \in \mathbb{R}^{300} \tag{33}$$
$$\mathbf{v}_{\text{Germany}} \in \mathbb{R}^{300} \tag{34}$$

3. Compute query vector:

$$\mathbf{q} = \mathbf{v}_{\text{Paris}} - \mathbf{v}_{\text{France}} + \mathbf{v}_{\text{Germany}} \tag{35}$$

4. For all words w in vocabulary (excluding input words), compute:

$$s(w) = \frac{\mathbf{q} \cdot \mathbf{v}_w}{\|\mathbf{q}\|\|\mathbf{v}_w\|} \tag{36}$$

5. Find word with maximum similarity:

$$w^* = \arg \max_{w \in V \setminus \{\text{Paris, France, Germany}\}} s(w) \tag{37}$$

Typical results (top 5 similar words):

$$\text{Berlin} : 0.74 \tag{38}$$
$$\text{Munich} : 0.68 \tag{39}$$
$$\text{Hamburg} : 0.64 \tag{40}$$
$$\text{Vienna} : 0.62 \tag{41}$$
$$\text{Dresden} : 0.60 \tag{42}$$

Answer: Berlin (highest similarity)

3 Linear Independence in Feature Engineering

Problem 3.1 (Multicollinearity Detection)**.** A machine learning model for house price prediction uses the following features:

$$x_1 : \text{Area in square feet} \tag{43}$$
$$x_2 : \text{Area in square meters} \tag{44}$$
$$x_3 : \text{Number of rooms} \tag{45}$$
$$x_4 : \text{Age of house (years)} \tag{46}$$

Known relationship: $x_2 = 0.0929 \times x_1$ (conversion factor: 1 sq.ft = 0.0929 sq.m)
Dataset sample:

$$X = \begin{bmatrix} 2000 & 185.8 & 3 & 10 \\ 1500 & 139.35 & 2 & 5 \\ 2500 & 232.25 & 4 & 15 \\ 1800 & 167.22 & 3 & 8 \end{bmatrix} \tag{47}$$

Questions:

1. Are the feature vectors linearly independent?
2. What is the rank of the feature matrix?
3. How does this affect regression?

Solution 3.1 (Detecting Linear Dependence)**. Step 1: Check for linear dependence**
We know: $x_2 = 0.0929x_1$
So feature vectors satisfy:

$$\mathbf{x}_2 = 0.0929\mathbf{x}_1 \tag{48}$$

This is a linear combination:

$$-0.0929\mathbf{x}_1 + 1\mathbf{x}_2 + 0\mathbf{x}_3 + 0\mathbf{x}_4 = \mathbf{0} \tag{49}$$

Conclusion: Features are linearly dependent!
Step 2: Compute rank using row reduction
Transpose feature matrix:

$$X^T = \begin{bmatrix} 2000 & 1500 & 2500 & 1800 \\ 185.8 & 139.35 & 232.25 & 167.22 \\ 3 & 2 & 4 & 3 \\ 10 & 5 & 15 & 8 \end{bmatrix} \tag{50}$$

Apply row reduction (eliminating R_2 using R_1):

$$R_2' = R_2 - 0.0929R_1 \tag{51}$$

$$R_2' = [185.8, 139.35, 232.25, 167.22] - 0.0929[2000, 1500, 2500, 1800] \tag{52}$$
$$= [185.8, 139.35, 232.25, 167.22] - [185.8, 139.35, 232.25, 167.22] \tag{53}$$
$$= [0, 0, 0, 0] \tag{54}$$

RREF:

$$\begin{bmatrix} 1 & * & * & * \\ 0 & 0 & 0 & 0 \\ 0 & * & * & * \\ 0 & * & * & * \end{bmatrix} \tag{55}$$

Rank = 3 (only 3 pivot columns)
Step 3: Impact on regression
For linear regression: $\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\epsilon}$
Normal equations:

$$(X^T X)\boldsymbol{\beta} = X^T \mathbf{y} \tag{56}$$

When features are dependent, $X^T X$ is **singular** (non-invertible):

$$\det(X^T X) = 0 \tag{57}$$

Consequences:

- Infinite solutions for $\boldsymbol{\beta}$
- Regression coefficients are unstable
- High standard errors
- Cannot use formula: $\boldsymbol{\beta} = (X^T X)^{-1} X^T \mathbf{y}$

Solution: Remove redundant feature (x_2), use regularization (Ridge/Lasso), or apply PCA.

VIF Calculation

Variance Inflation Factor measures multicollinearity:
For feature j:

$$\text{VIF}_j = \frac{1}{1 - R_j^2} \tag{58}$$

where R_j^2 is the R-squared from regressing x_j on all other features.
For our example:
Regressing x_2 on x_1: $x_2 = 0.0929 x_1$
Since relationship is exact: $R_2^2 = 1.0$

$$\text{VIF}_2 = \frac{1}{1 - 1} = \infty \tag{59}$$

Rule: VIF ¿ 10 indicates problematic multicollinearity

4 Dimensionality and Curse of Dimensionality

Problem 4.1 (K-Nearest Neighbors Performance)**.** A KNN classifier is trained on datasets of varying dimensions.

Setup:

- Training samples: $n = 1000$
- Test query point: $\mathbf{q}$
- Find $k = 5$ nearest neighbors

Datasets:

1. D_1: 1000 points in $\mathbb{R}^2$ (2D)
2. D_2: 1000 points in $\mathbb{R}^{10}$ (10D)
3. D_3: 1000 points in $\mathbb{R}^{100}$ (100D)
4. D_4: 1000 points in $\mathbb{R}^{1000}$ (1000D)

Question: How does distance to nearest neighbor change with dimension?

Solution 4.1 (Analyzing Curse of Dimensionality)**. Key Insight:** In high dimensions, all points become approximately equidistant.

Volume of hypersphere:
In d dimensions, volume of unit hypersphere:

$$V_d = \frac{\pi^{d/2}}{\Gamma(d/2+1)} \tag{60}$$

Distance concentration:
For random points in unit hypercube $[0,1]^d$:
Expected distance to nearest neighbor:

$$E[d_{\min}] \approx \left(\frac{1}{n}\right)^{1/d} \tag{61}$$

Calculations for n=1000:

$$d = 2: \quad E[d_{\min}] \approx (1/1000)^{1/2} = 0.0316 \tag{62}$$
$$d = 10: \quad E[d_{\min}] \approx (1/1000)^{1/10} = 0.501 \tag{63}$$
$$d = 100: \quad E[d_{\min}] \approx (1/1000)^{1/100} = 0.933 \tag{64}$$
$$d = 1000: \quad E[d_{\min}] \approx (1/1000)^{1/1000} = 0.993 \tag{65}$$

Ratio analysis:
Ratio of maximum to minimum distance to query point:

$$r_d = \frac{d_{\max} - d_{\min}}{d_{\min}} \tag{66}$$

As $d \to \infty$:

$$\lim_{d\to\infty} r_d = 0 \tag{67}$$

Practical implication:
In $\mathbb{R}^{1000}$, if nearest neighbor is at distance 0.993, and farthest is at 1.007:

$$r_{1000} = \frac{1.007 - 0.993}{0.993} \approx 0.014 \tag{68}$$

All points are nearly the same distance away! KNN becomes useless.
Sample requirement:
To maintain same density in higher dimensions:

$$n_{\text{required}} \propto 2^d \tag{69}$$

For $d = 10$: need $2^{10} = 1024$ samples
For $d = 100$: need $2^{100} \approx 1.27 \times 10^{30}$ samples (impossible!)

Numerical Example

Volume concentration in high dimensions:
Consider unit hypercube $[0, 1]^d$ and inscribed hypersphere of radius 0.5.
Volume ratio:

$$\frac{V_{\text{sphere}}}{V_{\text{cube}}} = \frac{\pi^{d/2}}{2^d \Gamma(d/2 + 1)} \tag{70}$$

Results:

$$d = 2: \quad \frac{\pi(0.5)^2}{1} = 0.785 \quad (78.5\% \text{ of volume}) \tag{71}$$

$$d = 5: \quad \frac{\pi^{2.5}(0.5)^5}{120} = 0.164 \quad (16.4\% \text{ of volume}) \tag{72}$$

$$d = 10: \quad \approx 0.0025 \quad (0.25\% \text{ of volume}) \tag{73}$$

$$d = 20: \quad \approx 2.46 \times 10^{-8} \quad (0.0000025\% \text{ of volume}) \tag{74}$$

Conclusion: In high dimensions, almost all volume is in the corners!

5 Neural Network Parameter Space

Problem 5.1 (Parameter Count and Vector Space Dimension)**.** Consider a simple neural network for MNIST digit classification:

Architecture:

- Input layer: 784 neurons (28×28 pixels)
- Hidden layer 1: 128 neurons (ReLU activation)
- Hidden layer 2: 64 neurons (ReLU activation)
- Output layer: 10 neurons (Softmax)

Questions:

1. What is the dimension of the parameter space?
2. Write the parameter vector explicitly
3. How many parameters need to be optimized?

Solution 5.1 (Computing Parameter Space Dimension)**. Layer 1: Input → Hidden1**
Weight matrix: $W^{(1)} \in \mathbb{R}^{784\times128}$
Number of weights: $784 \times 128 = 100,352$
Bias vector: $\mathbf{b}^{(1)} \in \mathbb{R}^{128}$
Number of biases: 128
Total for Layer 1: $100,352 + 128 = 100,480$
Layer 2: Hidden1 → Hidden2
Weight matrix: $W^{(2)} \in \mathbb{R}^{128\times64}$
Number of weights: $128 \times 64 = 8,192$
Bias vector: $\mathbf{b}^{(2)} \in \mathbb{R}^{64}$
Number of biases: 64
Total for Layer 2: $8,192 + 64 = 8,256$
Layer 3: Hidden2 → Output
Weight matrix: $W^{(3)} \in \mathbb{R}^{64\times10}$
Number of weights: $64 \times 10 = 640$
Bias vector: $\mathbf{b}^{(3)} \in \mathbb{R}^{10}$
Number of biases: 10
Total for Layer 3: $640 + 10 = 650$
Total Parameter Count:

$$p = 100,480 + 8,256 + 650 = 109,386 \tag{75}$$

Parameter Vector:
All parameters can be flattened into a single vector:

$$\boldsymbol{\theta} = [\text{vec}(W^{(1)}); \mathbf{b}^{(1)}; \text{vec}(W^{(2)}); \mathbf{b}^{(2)}; \text{vec}(W^{(3)}); \mathbf{b}^{(3)}] \in \mathbb{R}^{109386} \tag{76}$$

where $\text{vec}(\cdot)$ flattens a matrix into a column vector.
Parameter Space:
The parameter space is:

$$\Theta = \mathbb{R}^{109386} \tag{77}$$

This is a 109,386-dimensional vector space!

Gradient Descent Update:

At iteration t:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t) \tag{78}$$

where $\nabla_{\boldsymbol{\theta}} L \in \mathbb{R}^{109386}$ is the gradient vector.

ResNet-50 Parameter Count

Modern Deep Networks:

ResNet-50 for ImageNet classification:

- Input: $224 \times 224 \times 3$ images
- Total parameters: 25,557,032
- Parameter space: $\Theta \in \mathbb{R}^{25,557,032}$

GPT-3:

- Total parameters: 175 billion
- Parameter space: $\Theta \in \mathbb{R}^{175,000,000,000}$

Training navigates through these incredibly high-dimensional spaces!

6 Cosine Similarity in Recommender Systems

Problem 6.1 (Movie Recommendation)**.** A movie streaming service represents users and movies as vectors based on viewing history and genres.

User vectors (based on genre preferences):

$$\mathbf{u}_1 = [5, 1, 0, 4, 2] \quad \text{(User Alice)} \tag{79}$$
$$\mathbf{u}_2 = [1, 5, 4, 0, 1] \quad \text{(User Bob)} \tag{80}$$
$$\mathbf{u}_3 = [4, 2, 1, 3, 2] \quad \text{(User Charlie)} \tag{81}$$

Movie vectors (genre: [Action, Romance, Comedy, Thriller, Sci-Fi]):

$$\mathbf{m}_1 = [5, 0, 0, 5, 3] \quad \text{(Inception)} \tag{82}$$
$$\mathbf{m}_2 = [0, 5, 4, 0, 0] \quad \text{(The Notebook)} \tag{83}$$
$$\mathbf{m}_3 = [3, 1, 2, 4, 1] \quad \text{(Die Hard)} \tag{84}$$

Task: Recommend a movie to Alice using cosine similarity.

Solution 6.1 (Cosine Similarity Calculation)**. Formula:**

$$\text{cosine_sim}(\mathbf{u}, \mathbf{m}) = \frac{\mathbf{u} \cdot \mathbf{m}}{\|\mathbf{u}\|\|\mathbf{m}\|} \tag{85}$$

For Alice ($\mathbf{u}_1$) and Inception ($\mathbf{m}_1$):
Dot product:

$$\mathbf{u}_1 \cdot \mathbf{m}_1 = 5(5) + 1(0) + 0(0) + 4(5) + 2(3) \tag{86}$$
$$= 25 + 0 + 0 + 20 + 6 = 51 \tag{87}$$

Norms:

$$\|\mathbf{u}_1\| = \sqrt{5^2 + 1^2 + 0^2 + 4^2 + 2^2} \tag{88}$$
$$= \sqrt{25 + 1 + 0 + 16 + 4} = \sqrt{46} \approx 6.78 \tag{89}$$

$$\|\mathbf{m}_1\| = \sqrt{5^2 + 0^2 + 0^2 + 5^2 + 3^2} \tag{90}$$
$$= \sqrt{25 + 0 + 0 + 25 + 9} = \sqrt{59} \approx 7.68 \tag{91}$$

Cosine similarity:

$$\text{sim}(\mathbf{u}_1, \mathbf{m}_1) = \frac{51}{6.78 \times 7.68} = \frac{51}{52.07} \approx 0.980 \tag{92}$$

For Alice and The Notebook ($\mathbf{m}_2$):

$$\mathbf{u}_1 \cdot \mathbf{m}_2 = 5(0) + 1(5) + 0(4) + 4(0) + 2(0) \tag{93}$$
$$= 0 + 5 + 0 + 0 + 0 = 5 \tag{94}$$

$$\|\mathbf{m}_2\| = \sqrt{0^2 + 5^2 + 4^2 + 0^2 + 0^2} \tag{95}$$
$$= \sqrt{0 + 25 + 16 + 0 + 0} = \sqrt{41} \approx 6.40 \tag{96}$$

$$\text{sim}(\mathbf{u}_1, \mathbf{m}_2) = \frac{5}{6.78 \times 6.40} = \frac{5}{43.39} \approx 0.115 \tag{97}$$

For Alice and Die Hard ($\mathbf{m}_3$):

$$\mathbf{u}_1 \cdot \mathbf{m}_3 = 5(3) + 1(1) + 0(2) + 4(4) + 2(1) \tag{98}$$
$$= 15 + 1 + 0 + 16 + 2 = 34 \tag{99}$$

$$\|\mathbf{m}_3\| = \sqrt{3^2 + 1^2 + 2^2 + 4^2 + 1^2} \tag{100}$$
$$= \sqrt{9 + 1 + 4 + 16 + 1} = \sqrt{31} \approx 5.57 \tag{101}$$

$$\text{sim}(\mathbf{u}_1, \mathbf{m}_3) = \frac{34}{6.78 \times 5.57} = \frac{34}{37.77} \approx 0.900 \tag{102}$$

Ranking:

$$\text{Inception:} \quad 0.980 \tag{103}$$
$$\text{Die Hard:} \quad 0.900 \tag{104}$$
$$\text{The Notebook:} \quad 0.115 \tag{105}$$

Recommendation: Suggest "Inception" to Alice (highest similarity)

7 Subspaces in Image Processing

Problem 7.1 (Face Recognition Eigenfaces)**.** A face recognition system uses eigenfaces (PCA on face images).

Given:

- 400 training face images
- Each image: 100×100 pixels (grayscale)
- Each image vectorized: $\mathbf{x}_i \in \mathbb{R}^{10000}$

After PCA:

- Top 50 eigenfaces capture 95% variance
- Eigenfaces: $\{\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_{50}\}$

Question: What is the dimension of the face subspace? How do you project a new face onto this subspace?

Solution 7.1 (Face Subspace Projection)**. Face Subspace:**

The face subspace V_{face} is spanned by the top 50 eigenfaces:

$$V_{\text{face}} = \text{span}\{\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_{50}\} \subset \mathbb{R}^{10000} \tag{106}$$

Dimension:

$$\dim(V_{\text{face}}) = 50 \tag{107}$$

This is a 50-dimensional subspace within $\mathbb{R}^{10000}$.

Basis for subspace:

Form projection matrix:

$$E = [\mathbf{e}_1 \mid \mathbf{e}_2 \mid \cdots \mid \mathbf{e}_{50}] \in \mathbb{R}^{10000 \times 50} \tag{108}$$

where $\mathbf{e}_i^T \mathbf{e}_j = \delta_{ij}$ (orthonormal basis)

Projecting new face:

Given new face image $\mathbf{x}_{\text{new}} \in \mathbb{R}^{10000}$:

Step 1: Center the image

$$\tilde{\mathbf{x}}_{\text{new}} = \mathbf{x}_{\text{new}} - \boldsymbol{\mu} \tag{109}$$

where $\boldsymbol{\mu}$ is mean face from training set.

Step 2: Project onto face subspace

$$\mathbf{w} = E^T \tilde{\mathbf{x}}_{\text{new}} \in \mathbb{R}^{50} \tag{110}$$

This gives coordinates in the face subspace:

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_{50} \end{bmatrix} \tag{111}$$

where w_i is the coefficient for eigenface $\mathbf{e}_i$.

Step 3: Reconstruct approximation

$$\hat{\mathbf{x}}_{\text{new}} = \boldsymbol{\mu} + E\mathbf{w} \tag{112}$$

Recognition:
For each person p in database with representation $\mathbf{w}_p \in \mathbb{R}^{50}$:
Compute distance:

$$d_p = \|\mathbf{w} - \mathbf{w}_p\|_2 \tag{113}$$

Identify as person with minimum distance:

$$p^* = \arg\min_p d_p \tag{114}$$

Numerical Example

Simplified calculation with 3 eigenfaces in 2D:
Mean face:

$$\boldsymbol{\mu} = \begin{bmatrix} 128 \\ 128 \\ 128 \\ 128 \end{bmatrix} \quad \text{(gray pixels)} \tag{115}$$

Top 3 eigenfaces (simplified):

$$\mathbf{e}_1 = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0.5 \\ -0.5 \\ 0.5 \\ -0.5 \end{bmatrix}, \quad \mathbf{e}_3 = \begin{bmatrix} 0.5 \\ 0.5 \\ -0.5 \\ -0.5 \end{bmatrix} \tag{116}$$

New face:

$$\mathbf{x}_{\text{new}} = \begin{bmatrix} 200 \\ 150 \\ 180 \\ 130 \end{bmatrix} \tag{117}$$

Centered:

$$\tilde{\mathbf{x}}_{\text{new}} = \begin{bmatrix} 72 \\ 22 \\ 52 \\ 2 \end{bmatrix} \tag{118}$$

Coefficients:

$$w_1 = \mathbf{e}_1^T \tilde{\mathbf{x}}_{\text{new}} = 0.5(72 + 22 + 52 + 2) = 74 \tag{119}$$

$$w_2 = \mathbf{e}_2^T \tilde{\mathbf{x}}_{\text{new}} = 0.5(72 - 22 + 52 - 2) = 50 \tag{120}$$

$$w_3 = \mathbf{e}_3^T \tilde{\mathbf{x}}_{\text{new}} = 0.5(72 + 22 - 52 - 2) = 20 \tag{121}$$

Face representation: $\mathbf{w} = [74, 50, 20]^T \in \mathbb{R}^3$

8 Orthogonality in Signal Processing

Problem 8.1 (Fourier Transform as Change of Basis)**.** A discrete signal of length $N = 4$ can be represented in two bases:

Time domain basis (standard):

$$\mathcal{B}_{\text{time}} = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right\} \tag{122}$$

Frequency domain basis (Fourier):
DFT matrix for $N = 4$:

$$F = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix} \tag{123}$$

Signal: $\mathbf{x} = [1, 2, 3, 4]^T$
Task: Transform to frequency domain and interpret.

Solution 8.1 (Discrete Fourier Transform)**. DFT Formula:**
For signal $\mathbf{x} \in \mathbb{R}^N$:

$$\mathbf{X} = F\mathbf{x} \tag{124}$$

where F is the DFT matrix:

$$F_{kn} = \frac{1}{\sqrt{N}} e^{-2\pi ikn/N} \tag{125}$$

For N=4:

$$\mathbf{X} = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \tag{126}$$

Component calculations:
DC component ($k = 0$):

$$X_0 = \frac{1}{2}(1 + 2 + 3 + 4) = \frac{10}{2} = 5 \tag{127}$$

First frequency ($k = 1$):

$$X_1 = \frac{1}{2}(1 \cdot 1 + 2 \cdot (-i) + 3 \cdot (-1) + 4 \cdot i) \tag{128}$$

$$= \frac{1}{2}(1 - 2i - 3 + 4i) \tag{129}$$

$$= \frac{1}{2}(-2 + 2i) = -1 + i \tag{130}$$

Second frequency ($k = 2$):

$$X_2 = \frac{1}{2}(1 \cdot 1 + 2 \cdot (-1) + 3 \cdot 1 + 4 \cdot (-1)) \tag{131}$$

$$= \frac{1}{2}(1 - 2 + 3 - 4) = \frac{-2}{2} = -1 \tag{132}$$

Third frequency ($k = 3$):

$$X_3 = \frac{1}{2}(1 \cdot 1 + 2 \cdot i + 3 \cdot (-1) + 4 \cdot (-i)) \tag{133}$$

$$= \frac{1}{2}(1 + 2i - 3 - 4i) \tag{134}$$

$$= \frac{1}{2}(-2 - 2i) = -1 - i \tag{135}$$

Frequency representation:

$$\mathbf{X} = \begin{bmatrix} 5 \\ -1 + i \\ -1 \\ -1 - i \end{bmatrix} \tag{136}$$

Magnitude spectrum:

$$|X_0| = 5 \tag{137}$$

$$|X_1| = \sqrt{(-1)^2 + 1^2} = \sqrt{2} \tag{138}$$

$$|X_2| = 1 \tag{139}$$

$$|X_3| = \sqrt{(-1)^2 + (-1)^2} = \sqrt{2} \tag{140}$$

Inverse DFT (reconstruction):

$$\mathbf{x} = F^* \mathbf{X} \tag{141}$$

where $F^* = F^H$ (conjugate transpose) for unitary DFT matrix.
Verification:

$$FF^* = I_4 \tag{142}$$

The DFT basis vectors are orthonormal!

9 Rank and Column Space in Data Analysis

Problem 9.1 (Image Compression via Low-Rank Approximation)**.** A grayscale image is represented as a matrix $A \in \mathbb{R}^{512\times512}$.

Singular Value Decomposition:

$$A = U\Sigma V^T \tag{143}$$

where:

- $U \in \mathbb{R}^{512\times512}$ (left singular vectors)
- $\Sigma \in \mathbb{R}^{512\times512}$ (diagonal with singular values)
- $V \in \mathbb{R}^{512\times512}$ (right singular vectors)

Singular values: $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{512}$
Top 50 singular values capture 99% of energy:

$$\frac{\sum_{i=1}^{50}\sigma_i^2}{\sum_{i=1}^{512}\sigma_i^2} = 0.99 \tag{144}$$

Task: Compute rank-50 approximation and compression ratio.

Solution 9.1 (Low-Rank Approximation)**. Rank-k approximation:**

$$A_k = \sum_{i=1}^{k}\sigma_i\mathbf{u}_i\mathbf{v}_i^T \tag{145}$$

For $k = 50$:

$$A_{50} = \sigma_1\mathbf{u}_1\mathbf{v}_1^T + \sigma_2\mathbf{u}_2\mathbf{v}_2^T + \cdots + \sigma_{50}\mathbf{u}_{50}\mathbf{v}_{50}^T \tag{146}$$

Matrix form:

$$A_{50} = U_{50}\Sigma_{50}V_{50}^T \tag{147}$$

where:

$$U_{50} = [\mathbf{u}_1 \mid \mathbf{u}_2 \mid \cdots \mid \mathbf{u}_{50}] \in \mathbb{R}^{512\times50} \tag{148}$$
$$\Sigma_{50} = \mathrm{diag}(\sigma_1, \sigma_2, \ldots, \sigma_{50}) \in \mathbb{R}^{50\times50} \tag{149}$$
$$V_{50} = [\mathbf{v}_1 \mid \mathbf{v}_2 \mid \cdots \mid \mathbf{v}_{50}] \in \mathbb{R}^{512\times50} \tag{150}$$

Storage requirement:
Original image:

$$512 \times 512 = 262,144 \text{ values} \tag{151}$$

Rank-50 approximation:

$$U_{50}: \quad 512 \times 50 = 25,600 \text{ values} \tag{152}$$
$$\Sigma_{50}: \quad 50 \text{ values} \tag{153}$$
$$V_{50}: \quad 512 \times 50 = 25,600 \text{ values} \tag{154}$$

Total: $25,600 + 50 + 25,600 = 51,250$ values

Compression ratio:

$$\text{CR} = \frac{262,144}{51,250} \approx 5.11 \tag{155}$$

Image is compressed by factor of 5 while retaining 99% of information!

Approximation error (Frobenius norm):

$$\|A - A_{50}\|_F = \sqrt{\sum_{i=51}^{512} \sigma_i^2} \tag{156}$$

Relative error:

$$\frac{\|A - A_{50}\|_F}{\|A\|_F} = \sqrt{\frac{\sum_{i=51}^{512} \sigma_i^2}{\sum_{i=1}^{512} \sigma_i^2}} = \sqrt{1 - 0.99} \approx 0.10 \tag{157}$$

Only 10% relative error!

Column Space Interpretation

Geometric meaning:

Original image A has column space:

$$\mathcal{C}(A) \subseteq \mathbb{R}^{512} \tag{158}$$

with $\dim(\mathcal{C}(A)) = \text{rank}(A) \leq 512$

Low-rank approximation A_{50} has:

$$\mathcal{C}(A_{50}) = \text{span}\{\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_{50}\} \tag{159}$$

with $\dim(\mathcal{C}(A_{50})) = 50$

Each column of A_{50} lies in a 50-dimensional subspace of $\mathbb{R}^{512}$!

Best approximation theorem:

Among all rank-k matrices B:

$$A_k = \arg \min_{\text{rank}(B)=k} \|A - B\|_F \tag{160}$$

SVD gives the optimal solution!

10 Gram-Schmidt Orthogonalization in QR Decomposition

Problem 10.1 (Creating Orthonormal Features)**.** A dataset has 3 correlated features:

$$X = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \tag{161}$$

Task: Apply Gram-Schmidt to create orthonormal basis for column space.

Solution 10.1 (Gram-Schmidt Orthogonalization)**. Given vectors:**

$$\mathbf{a}_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \quad \mathbf{a}_2 = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 2 \end{bmatrix}, \quad \mathbf{a}_3 = \begin{bmatrix} 2 \\ 1 \\ 1 \\ 3 \end{bmatrix} \tag{162}$$

Step 1: Normalize first vector

$$\mathbf{q}_1 = \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|} \tag{163}$$

$$\|\mathbf{a}_1\| = \sqrt{1^2 + 1^2 + 0^2 + 1^2} = \sqrt{3} \tag{164}$$

$$\mathbf{q}_1 = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.577 \\ 0.577 \\ 0 \\ 0.577 \end{bmatrix} \tag{165}$$

Step 2: Orthogonalize second vector
Remove component of $\mathbf{a}_2$ in direction of $\mathbf{q}_1$:

$$\mathbf{u}_2 = \mathbf{a}_2 - (\mathbf{a}_2 \cdot \mathbf{q}_1)\mathbf{q}_1 \tag{166}$$

$$\mathbf{a}_2 \cdot \mathbf{q}_1 = \frac{1}{\sqrt{3}}(1 \cdot 1 + 0 \cdot 1 + 1 \cdot 0 + 2 \cdot 1) \tag{167}$$

$$= \frac{3}{\sqrt{3}} = \sqrt{3} \tag{168}$$

$$\mathbf{u}_2 = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 2 \end{bmatrix} - \sqrt{3} \cdot \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} \tag{169}$$

$$= \begin{bmatrix} 1 \\ 0 \\ 1 \\ 2 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 1 \\ 1 \end{bmatrix} \tag{170}$$

Normalize:

$$\|\mathbf{u}_2\| = \sqrt{0^2 + (-1)^2 + 1^2 + 1^2} = \sqrt{3} \tag{171}$$

$$\mathbf{q}_2 = \frac{1}{\sqrt{3}} \begin{bmatrix} 0 \\ -1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -0.577 \\ 0.577 \\ 0.577 \end{bmatrix} \tag{172}$$

Step 3: Orthogonalize third vector

$$\mathbf{u}_3 = \mathbf{a}_3 - (\mathbf{a}_3 \cdot \mathbf{q}_1)\mathbf{q}_1 - (\mathbf{a}_3 \cdot \mathbf{q}_2)\mathbf{q}_2 \tag{173}$$

$$\mathbf{a}_3 \cdot \mathbf{q}_1 = \frac{1}{\sqrt{3}}(2 + 1 + 0 + 3) = \frac{6}{\sqrt{3}} = 2\sqrt{3} \tag{174}$$

$$\mathbf{a}_3 \cdot \mathbf{q}_2 = \frac{1}{\sqrt{3}}(0 - 1 + 1 + 3) = \frac{3}{\sqrt{3}} = \sqrt{3} \tag{175}$$

$$\mathbf{u}_3 = \begin{bmatrix} 2 \\ 1 \\ 1 \\ 3 \end{bmatrix} - 2\sqrt{3} \cdot \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} - \sqrt{3} \cdot \frac{1}{\sqrt{3}} \begin{bmatrix} 0 \\ -1 \\ 1 \\ 1 \end{bmatrix} \tag{176}$$

$$= \begin{bmatrix} 2 \\ 1 \\ 1 \\ 3 \end{bmatrix} - \begin{bmatrix} 2 \\ 2 \\ 0 \\ 2 \end{bmatrix} - \begin{bmatrix} 0 \\ -1 \\ 1 \\ 1 \end{bmatrix} \tag{177}$$

$$= \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \tag{178}$$

Result: $\mathbf{a}_3$ is in span of $\{\mathbf{a}_1, \mathbf{a}_2\}$!
Only 2 independent columns. Column space has dimension 2.
Orthonormal basis for column space:

$$Q = [\mathbf{q}_1 \mid \mathbf{q}_2] = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 0 \\ 1 & -1 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \tag{179}$$

Verification of orthonormality:

$$\mathbf{q}_1 \cdot \mathbf{q}_2 = \frac{1}{3}(0 - 1 + 0 + 1) = 0 \quad \checkmark \tag{180}$$

$$\mathbf{q}_1 \cdot \mathbf{q}_1 = \frac{1}{3}(1 + 1 + 0 + 1) = 1 \quad \checkmark \tag{181}$$

$$\mathbf{q}_2 \cdot \mathbf{q}_2 = \frac{1}{3}(0 + 1 + 1 + 1) = 1 \quad \checkmark \tag{182}$$

11 Kernel Methods and Implicit Feature Spaces

Problem 11.1 (RBF Kernel for Non-linear Classification)**.** Consider 2D data that is not linearly separable:

Training data:

$$\text{Class } +1: \quad \mathbf{x}_1 = [0,0]^T, \mathbf{x}_2 = [2,2]^T \tag{183}$$

$$\text{Class } -1: \quad \mathbf{x}_3 = [0,2]^T, \mathbf{x}_4 = [2,0]^T \tag{184}$$

RBF Kernel:

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right) \tag{185}$$

with $\sigma = 1$.

Task: Compute kernel (Gram) matrix and interpret as inner product in feature space.

Solution 11.1 (Kernel Matrix Computation)**. Kernel formula with $\sigma = 1$:**

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2}\right) \tag{186}$$

Compute pairwise distances:

$$\|\mathbf{x}_1 - \mathbf{x}_1\|^2 = 0 \tag{187}$$

$$\|\mathbf{x}_1 - \mathbf{x}_2\|^2 = (0-2)^2 + (0-2)^2 = 8 \tag{188}$$

$$\|\mathbf{x}_1 - \mathbf{x}_3\|^2 = (0-0)^2 + (0-2)^2 = 4 \tag{189}$$

$$\|\mathbf{x}_1 - \mathbf{x}_4\|^2 = (0-2)^2 + (0-0)^2 = 4 \tag{190}$$

$$\|\mathbf{x}_2 - \mathbf{x}_3\|^2 = (2-0)^2 + (2-2)^2 = 4 \tag{191}$$

$$\|\mathbf{x}_2 - \mathbf{x}_4\|^2 = (2-2)^2 + (2-0)^2 = 4 \tag{192}$$

$$\|\mathbf{x}_3 - \mathbf{x}_4\|^2 = (0-2)^2 + (2-0)^2 = 8 \tag{193}$$

Compute kernel values:

$$k(\mathbf{x}_1, \mathbf{x}_1) = e^0 = 1 \tag{194}$$

$$k(\mathbf{x}_1, \mathbf{x}_2) = e^{-8/2} = e^{-4} \approx 0.0183 \tag{195}$$

$$k(\mathbf{x}_1, \mathbf{x}_3) = e^{-4/2} = e^{-2} \approx 0.1353 \tag{196}$$

$$k(\mathbf{x}_1, \mathbf{x}_4) = e^{-4/2} = e^{-2} \approx 0.1353 \tag{197}$$

$$k(\mathbf{x}_2, \mathbf{x}_3) = e^{-2} \approx 0.1353 \tag{198}$$

$$k(\mathbf{x}_2, \mathbf{x}_4) = e^{-2} \approx 0.1353 \tag{199}$$

$$k(\mathbf{x}_3, \mathbf{x}_4) = e^{-4} \approx 0.0183 \tag{200}$$

Gram matrix:

$$K = \begin{bmatrix} 1 & 0.0183 & 0.1353 & 0.1353 \\ 0.0183 & 1 & 0.1353 & 0.1353 \\ 0.1353 & 0.1353 & 1 & 0.0183 \\ 0.1353 & 0.1353 & 0.0183 & 1 \end{bmatrix} \tag{201}$$

Interpretation:
RBF kernel implicitly maps to infinite-dimensional Hilbert space $\mathcal{H}$:

$$\phi : \mathbb{R}^2 \to \mathcal{H} \tag{202}$$

where:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle_{\mathcal{H}} \tag{203}$$

Pattern recognition:
Same-class similarities:

$$k(\mathbf{x}_1, \mathbf{x}_2) = 0.0183 \quad \text{(both class +1)} \tag{204}$$
$$k(\mathbf{x}_3, \mathbf{x}_4) = 0.0183 \quad \text{(both class -1)} \tag{205}$$

Different-class similarities:

$$k(\mathbf{x}_1, \mathbf{x}_3) = k(\mathbf{x}_1, \mathbf{x}_4) = k(\mathbf{x}_2, \mathbf{x}_3) = k(\mathbf{x}_2, \mathbf{x}_4) = 0.1353 \tag{206}$$

Despite being linearly inseparable in $\mathbb{R}^2$, data may be separable in $\mathcal{H}$!
Decision function in feature space:

$$f(\mathbf{x}) = \sum_{i=1}^{4} \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b \tag{207}$$

where α_i are learned by SVM optimization.

12 Batch Normalization and Affine Transformations

Problem 12.1 (Batch Normalization Layer)**.** A batch normalization layer in a neural network processes a mini-batch of 3 samples with 4 features each.

Input batch:

$$X = \begin{bmatrix} 2 & 4 & 1 & 3 \\ 5 & 7 & 4 & 6 \\ 8 & 10 & 7 & 9 \end{bmatrix} \in \mathbb{R}^{3\times 4} \tag{208}$$

Task: Compute batch-normalized output.

Solution 12.1 (Batch Normalization Computation)**. Step 1: Compute mean for each feature**

$$\boldsymbol{\mu} = \frac{1}{m}\sum_{i=1}^{m} \mathbf{x}_i \tag{209}$$

where $m = 3$ (batch size).

$$\mu_1 = \frac{2+5+8}{3} = \frac{15}{3} = 5 \tag{210}$$

$$\mu_2 = \frac{4+7+10}{3} = \frac{21}{3} = 7 \tag{211}$$

$$\mu_3 = \frac{1+4+7}{3} = \frac{12}{3} = 4 \tag{212}$$

$$\mu_4 = \frac{3+6+9}{3} = \frac{18}{3} = 6 \tag{213}$$

$$\boldsymbol{\mu} = [5, 7, 4, 6]^T \tag{214}$$

Step 2: Compute variance for each feature

$$\sigma_j^2 = \frac{1}{m}\sum_{i=1}^{m}(x_{ij} - \mu_j)^2 \tag{215}$$

$$\sigma_1^2 = \frac{(2-5)^2 + (5-5)^2 + (8-5)^2}{3} = \frac{9+0+9}{3} = 6 \tag{216}$$

$$\sigma_2^2 = \frac{(4-7)^2 + (7-7)^2 + (10-7)^2}{3} = \frac{9+0+9}{3} = 6 \tag{217}$$

$$\sigma_3^2 = \frac{(1-4)^2 + (4-4)^2 + (7-4)^2}{3} = \frac{9+0+9}{3} = 6 \tag{218}$$

$$\sigma_4^2 = \frac{(3-6)^2 + (6-6)^2 + (9-6)^2}{3} = \frac{9+0+9}{3} = 6 \tag{219}$$

$$\boldsymbol{\sigma}^2 = [6, 6, 6, 6]^T \tag{220}$$

Step 3: Normalize

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \tag{221}$$

Using $\epsilon = 10^{-5}$ (for numerical stability):

$$\sqrt{\sigma_j^2 + \epsilon} \approx \sqrt{6} \approx 2.449 \tag{222}$$

For first sample:

$$\hat{x}_{11} = \frac{2-5}{2.449} = \frac{-3}{2.449} \approx -1.225 \tag{223}$$

$$\hat{x}_{12} = \frac{4-7}{2.449} = \frac{-3}{2.449} \approx -1.225 \tag{224}$$

$$\hat{x}_{13} = \frac{1-4}{2.449} = \frac{-3}{2.449} \approx -1.225 \tag{225}$$

$$\hat{x}_{14} = \frac{3-6}{2.449} = \frac{-3}{2.449} \approx -1.225 \tag{226}$$

Similarly for all samples:

$$\hat{X} = \begin{bmatrix} -1.225 & -1.225 & -1.225 & -1.225 \\ 0 & 0 & 0 & 0 \\ 1.225 & 1.225 & 1.225 & 1.225 \end{bmatrix} \tag{227}$$

Step 4: Scale and shift (learnable parameters)

$$y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j \tag{228}$$

If $\boldsymbol{\gamma} = [2, 2, 2, 2]^T$ and $\boldsymbol{\beta} = [1, 1, 1, 1]^T$:

$$Y = \begin{bmatrix} -1.45 & -1.45 & -1.45 & -1.45 \\ 1 & 1 & 1 & 1 \\ 3.45 & 3.45 & 3.45 & 3.45 \end{bmatrix} \tag{229}$$

Properties verified:
For each feature j in normalized batch:

$$\text{mean}(\hat{X}_j) = 0 \tag{230}$$

$$\text{var}(\hat{X}_j) = 1 \tag{231}$$

Geometric interpretation:
Batch normalization:

1. Translates feature distribution to zero mean (centering in vector space)
2. Scales to unit variance (normalization in vector space)
3. Applies learnable affine transformation

This is a change of basis in feature space!

13 Additional Practice Problems

Problem 13.1 (Attention Mechanism as Similarity)**.** In transformer models, attention scores are computed using dot products.

Query: $\mathbf{q} = [1, 0, 1]^T$

Keys:

$$\mathbf{k}_1 = [1, 0, 0]^T \tag{232}$$

$$\mathbf{k}_2 = [0, 1, 0]^T \tag{233}$$

$$\mathbf{k}_3 = [0, 0, 1]^T \tag{234}$$

Task: Compute attention weights using softmax.

Formula:

$$\alpha_i = \frac{\exp(\mathbf{q}^T \mathbf{k}_i / \sqrt{d_k})}{\sum_{j=1}^{3} \exp(\mathbf{q}^T \mathbf{k}_j / \sqrt{d_k})} \tag{235}$$

where $d_k = 3$.

Problem 13.2 (t-SNE Dimensionality Reduction)**.** t-SNE reduces high-dimensional data to 2D/3D for visualization.

Given: 4 points in $\mathbb{R}^{10}$ with pairwise distances:

$$D = \begin{bmatrix} 0 & 2.5 & 5.0 & 4.8 \\ 2.5 & 0 & 4.9 & 5.1 \\ 5.0 & 4.9 & 0 & 2.6 \\ 4.8 & 5.1 & 2.6 & 0 \end{bmatrix} \tag{236}$$

Task: Compute similarity matrix using Gaussian kernel with $\sigma = 1$.

Problem 13.3 (Residual Connections in ResNet)**.** A residual block computes:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{237}$$

Input: $\mathbf{x} = [1, 2, 3]^T$

Transformation: $\mathcal{F}(\mathbf{x}) = W_2 \sigma(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$

where:

$$W_1 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \tag{238}$$

$$W_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b}_2 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{239}$$

σ is ReLU activation.

Task: Compute output $\mathbf{y}$.

14 Summary of Vector Space Concepts in ML/DL

Key Takeaways

1. **Dimensionality:**
   - Feature dimension = number of input features
   - Parameter dimension = total learnable parameters
   - Intrinsic dimension often much smaller (manifold hypothesis)
2. **Subspaces:**
   - PCA finds optimal linear subspace
   - Autoencoders find non-linear subspaces
   - Face space, word embedding space are subspaces
3. **Similarity Measures:**
   - Euclidean distance: L2 norm
   - Cosine similarity: angle between vectors
   - Kernel functions: implicit inner products
4. **Transformations:**
   - Linear layers: matrix multiplication
   - Batch norm: affine transformation
   - Attention: weighted combinations
5. **Optimization:**
   - Gradient descent in parameter space
   - Manifold of optimal solutions
   - Regularization constrains to subspace